

Threshold ECDSA in Two Rounds

Yingjie Lyu*

School of Cyber Science
and Technology
Shandong University
Qingdao, China
yingjielyu@mail.sdu.edu.cn

Zengpeng Li[†]

School of Cyber Science
and Technology
Shandong University
Qingdao, China
zengpengliz@gmail.com

Hong-Sheng Zhou

Virginia Commonwealth
University
Richmond, United States
hszhou@vcu.edu

Xudong Deng*

School of Cyber Science
and Technology
Shandong University
Qingdao, China
xudongdeng@mail.sdu.edu.cn

Abstract

We propose the first two-round multi-party signing protocol for the Elliptic Curve Digital Signature Algorithm (ECDSA) in the threshold-optimal setting, reducing the number of rounds by one compared to the state of the art (Doerner et al., S&P '24). We also resolve the security issue of presigning pointed out by Groth and Shoup (Eurocrypt '22), evading a security loss that increases with the number of pre-released, unused presignatures, for the first time among threshold-optimal schemes.

Our construction builds on Non-Interactive Multiplication (NIM), a notion proposed by Boyle et al. (PKC '25), which allows parties to evaluate multiplications on secret-shared values in one round. In particular, we use the construction of Abram et al. (Eurocrypt '24) instantiated with class groups. The setup is minimal and transparent, consisting of only two class-group generators. The signing protocol is efficient in bandwidth, with a message size of 1.9 KiB at 128-bit security, and has competitive computational performance.

CCS Concepts

• Security and privacy → Digital signatures.

Keywords

ECDSA; signatures; threshold cryptography; blockchain; cryptocurrencies

ACM Reference Format:

Yingjie Lyu, Zengpeng Li, Hong-Sheng Zhou, and Xudong Deng. 2025. Threshold ECDSA in Two Rounds. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3719027.3765176>

1 Introduction

In a (t, n) threshold signature scheme [30], the signing key is secret-shared among n parties, and to issue a signature, a subset of at least t parties have to collaboratively interact. Threshold signatures are especially useful in cryptocurrency wallets for the extra security they provide. To forge signatures, an adversary has to corrupt t parties, which is much harder than in the single-party case. In

recent years, threshold ECDSA has received extensive attention [38, 37, 46, 31, 16, 17, 36, 29, 58, 2, 8, 18, 55, 10, 57, 54, 32, 34, 47, 27, 28, 50, 39], as ECDSA is a long-standing NIST standard and is widely adopted across major blockchains.

The Quest for Fewer Rounds. We consider the threshold-optimal setting, where the threshold t can be chosen arbitrarily from 2 to n . In this most general case, the earliest practical constructions by Gennaro and Goldfeder [37] and Lindell and Nof [46] require eight rounds of interaction. This was later improved to four rounds by Canetti et al. [16], and more recently, a three-round protocol was introduced by Doerner et al. [32], which remains the state of the art. In contrast, simple and efficient *two-round* threshold Schnorr schemes are already known [44, 7], naturally prompting the question:

Is a practical two-round threshold ECDSA protocol possible?

Due to the inherent complexity of ECDSA, this question has remained unresolved. Recent advances by Boneh et al. [9] and Adjedj et al. [4] have demonstrated two-round protocols in the two-party setting, but extending these results to the general multi-party case remains an open challenge.

Handling Many Presigning Sessions Securely. Many threshold ECDSA schemes since [16, 28] leverage preprocessing to minimize signing latency. The signing protocol is split into an offline *presigning* phase – run before the message to be signed is known – and an online phase. Ideally, presigning should dominate the total cost, and the online phase should be fast and *non-interactive*: each signer sends one protocol message upon receiving the signing input, and the messages are publicly combined into the signature output.

However, an analysis by Groth and Shoup [40] shows that security degrades when too many presigning sessions are run in advance. Specifically, for each prematurely revealed R value (presignature), if the adversary can find two message hashes whose ratio hits one of polynomially many special values, it can forge a signature. As a result, the security that existing threshold ECDSA schemes with presigning can possibly achieve is worse than that of plain ECDSA. An additional assumption on the hash function must be introduced, which is not needed for plain ECDSA. Moreover, this weakness completely breaks any scheme that both performs presigning and admits signing raw message hashes at the same time.

Some mitigations have been incorporated in two-party [4] and honest-majority [39] schemes. However, in the threshold-optimal setting, existing schemes can only address this issue by limiting the number of presigning sessions, resulting in an unsatisfactory trade-off between security and efficiency. The full version of [32] warns that “presigning should only be used in practice by those

*Also with State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. CCS '25, Taipei

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3765176>

who understand and accept the implications and risks associated with it.” The performance cost of giving up presigning is high, and it would be preferable if one can implement presigning without such concerns.

Our Results. We present the first two-round threshold ECDSA protocol in the threshold-optimal, multi-party setting. This provides an affirmative answer to the motivating question, which is open since Doerner et al. [32] and recently highlighted by Boneh et al. [9]. Moreover, our protocol is the first in the threshold-optimal setting that is immune to the attack surface revealed in [40], reclaiming the full efficiency of presigning.

1.1 Technical Overview

We begin with essential background, followed by a high-level overview of our solution. We then discuss the efficiency of our scheme in comparison to prior work.

ECDSA. Let $\mathbb{G}$ be an elliptic curve group generated by g of prime order q . Let $x \in \mathbb{Z}_q$ be the signing key, and $X = g^x$ be the verification key. To sign a message, the SHA-2 hash of which is $m \in \mathbb{Z}_q$, one chooses a random nonce $k \in \mathbb{Z}_q$, computes $R = g^k$, and sets r to be the x -coordinate of R , i.e., $r := R|_{x\text{-axis}}$. The signature is (r, σ) where $\sigma = k^{-1}(m + rx) \bmod q$.

Baseline Threshold ECDSA. In threshold ECDSA schemes, both x and k are secret-shared among parties and cannot be known by anyone (since knowing k allows recovery of x). Computing σ thus requires multi-party computation of inversion and multiplication on shared secrets. This is more complex than in threshold Schnorr signatures, where secrets are simply linearly combined.

Our starting point is the scheme of Doerner et al. [32]. Let each party i hold an additive share x_i of the secret key $x = x_1 + \dots + x_t$ across a quorum of t parties. Signing proceeds in three rounds:

- **ROUND 1:** Each party i samples random k_i and γ_i , commits to g^{k_i} , and initiates Oblivious Linear Evaluation (OLE) with all others using input γ_i .
- **ROUND 2:** Party i responds to each OLE instance, so each ordered pair (i, j) obtains additive shares of $k_i\gamma_j$ and $x_i\gamma_j$. Parties each decommit g^{k_i} , making $R = g^k$ and $r = R|_{x\text{-axis}}$ public, where $k = \sum_i k_i$. Each party locally aggregates to obtain additive shares $(k\gamma)_i$ and $(x\gamma)_i$ of products $k\gamma$ and $x\gamma$, where $\gamma = \sum_i \gamma_i$.
- **ROUND 3:** Given message hash m , each party sends $m\gamma_i + r(x\gamma)_i$ and $(k\gamma)_i$. Combining them gives $\gamma(m + rx)$ and $k\gamma$, then dividing yields $\sigma = k^{-1}(m + rx)$.

Many schemes use OLE to share multiplication results. Typically, OLE is realized in two rounds using Homomorphic Encryption or Oblivious Transfer. In such a protocol, Bob inputs b in round one; Alice inputs a in round two; the output satisfies $c + d = ab$, with c and d held by Alice and Bob, respectively.

Non-Interactive Multiplication. Towards two-round threshold signing, we leverage a key tool stemming from Homomorphic Secret Sharing (HSS) [13]. Abram et al. [3] proposed Bilinear HSS, an elegant way for two parties to evaluate one multiplication on additively secret-shared values *in one round*. It also fits a later notion of Non-Interactive Multiplication (NIM) by Boyle et al. [12]. The construction is a descendant of the Non-Interactive Vector OLE

(NIVOLE) scheme in [48], which is possible thanks to a Distributed Discrete Logarithm mechanism found in Paillier groups [48, 51]; using ideal class groups further avoids trusted setup [1]. Using NIM in place of existing OLE schemes in threshold ECDSA could bring savings in round count and bandwidth, and its computational cost is on par with OLE from linearly-homomorphic encryption. Though proposed in a semi-honest setting, it can be upgraded to malicious security using proofs of knowledge.

However, a new tool realizing multiplication with one fewer round does not make two-round threshold ECDSA immediately possible. All prior schemes require a commitment to g^{k_i} in the first round, the equivocation of which allows the simulation to make R hit a designated value. If we collapse the first two rounds of [32] into one, removing the commitment, the resulting scheme is likely insecure, because a rushing adversary would be able to bias the distribution of R . Thus, provable security of the two-round design on the horizon becomes the main challenge.

Presignature Re-randomization to the Rescue. A key insight in [40] is that re-randomizing the presignature at signing time restores security to the level of plain ECDSA. This design is shared by recent two-round threshold Schnorr schemes [44, 7].

Another valuable lesson is that threshold ECDSA can be built from different assumptions modeling single-party ECDSA with different “modes of operation.” Several such assumptions [16, 40, 4] have been analyzed in the Generic Group Model (GGM). While GGM is a strong idealized model, such idealization is known to be required for ECDSA [41], and we can draw useful comparative conclusions from these analyses.

Adedj et al. [4] extend [40] by allowing an adversary to tweak the nonce multiplicatively before it is properly re-randomized additively. Their analysis shows only a constant-factor security loss compared to plain ECDSA, which is still significantly better than presigning without re-randomization [16]. Inspired by their idea, we are finally able to construct a provably-secure two-round scheme.

Our Construction. We sketch our two-round signing protocol:

- **ROUND 1:** Each party i samples random k_i , γ_i , publishes g^{k_i} , and inputs k_i , x_i , and γ_i into NIM. Each pair (i, j) obtains additive shares of $k_i\gamma_j$ and $x_i\gamma_j$. After the interaction, each party holds additive shares of $k\gamma$ and $x\gamma$, where $k = \sum_i k_i$ and $\gamma = \sum_i \gamma_i$. The value g^k is publicly known.
- **ROUND 2:** As the message to be signed is known, the nonce is re-randomized from k to $zk + y$, where z, y are hashes derived from the transcript. Also, $R = g^{zk+y}$ and $r = R|_{x\text{-axis}}$ are known. Each party i sends $m\gamma_i + r(x\gamma)_i$ and $z(k\gamma)_i + y\gamma_i$. Combining them gives $\gamma(m + rx)$ and $\gamma(zk + y)$, then dividing yields $\sigma = (zk + y)^{-1}(m + rx)$.

Provided with the secrets of corrupted parties, which are extracted from their proofs of knowledge, the simulator can program the random oracles producing the re-randomizers z and y in a suitable way, so that a reduction to the assumption analyzed in [4] is possible, assuming the security of the building blocks.

Efficiency. Our scheme outperforms the state of the art in terms of bandwidth efficiency (table 2). Each party communicates 1.9 KiB in the presigning phase and 64 bytes in the online phase, at the 128-bit security level. It only uses 4% as much communication as

the scheme of Doerner et al. [32], and 10% as much as the scheme of Canetti et al. [16], in peer-to-peer networks. If parties are connected via centralized servers, our advantage increases further, as each party's outbound communication is constant for ours but linear in the number of signers for most other schemes in this setting.

We implement our scheme and compare it with some prior schemes in computational cost. The one of Doerner et al. [32] is very lightweight in computation, as it builds on OT extension and for the most part only performs symmetric-cryptographic operations. However, even though our presigning is about 10x heavier in computation than [32] and takes about 200 milliseconds CPU time for 3 parties, our scheme also enjoys a fast online phase taking < 1 millisecond of computation. Since our protocol is about 4x faster than [16], which has seen particular industrial interest, we consider our proposed scheme to be competitive in practical performance. As class groups find increasing use in cryptography, we expect that our scheme will benefit from future optimizations in this area.

1.2 Related Work

We compare the basic features of existing practical threshold ECDSA schemes in the threshold-optimal setting in table 1.

Table 1: Overview of threshold-optimal ECDSA schemes.

Scheme	Main Tool	Rounds	Note
GG18 [37]	Paillier	8	BC
LN18 [46]	Paillier / OT	8	original ver.
	OT	5	updated ver.
DKLs19 [31]	OT	$\lceil \log t \rceil + 6$	BC
CGGMP20 [16]	Paillier	4 or 7	BC, IA
CCLST20 [17]	CL	8	BC
CCLST23 [18]	CL	7	BC, IA
WMYC23 [55]	CL	7	BC, IA, robust
DKLs24 [32]	OT	3	
WMC24 [54]	threshold CL	4	BC, IA, robust
CDKS24 [25]	OT	7	BC, IA
This work	class group NIM	2	

IA = Identifiable Abort. BC = Requires broadcast channel during signing (if IA is not required, the cheaper *echo broadcast* can be used instead). OT = Oblivious Transfer. CL = Castagnos–Laguillaumie encryption. PCG = Pseudorandom Correlated Generator. NIM = Non-Interactive Multiplication.

Gennaro and Goldfeder [37] and Lindell and Nof [46] proposed the first practical threshold-optimal ECDSA schemes. Both constructions use two-party secure computation subprotocols, based on Paillier encryption or Oblivious Transfer (OT), to convert a product of shared secrets to additive shares.

Canetti et al. [16] extend [37] to support identifiable abort, i.e., identification of cheating parties in the case of signing abort, using NIZK arguments. Their protocol takes four rounds, one of which is a broadcast. In fact, broadcasts are necessary for identifiable abort with a dishonest majority [26]. To avoid broadcast overhead, we assume no broadcasts during signing, and our scheme does not have identifiable abort.

Castagnos et al. [17] use the Castagnos–Laguillaumie (CL) encryption scheme [20] from class groups in place of Paillier to achieve

lower bandwidth and avoid range proofs used in [37]. They later extend it to support identifiable abort [18]. Wong et al. [55] utilize Distributed Key Generation to generate Shamir secret-shared nonces, so that faults do not always abort signing. Later, Wong et al. [54] construct threshold ECDSA from threshold CL encryption [15]. This revives the idea of Gennaro et al. [38] using threshold Paillier, which is impractical due to a highly expensive setup [23]. The drawback of [54] is higher latency in the online phase.

Abram et al. [2] proposed a low-bandwidth construction using a Pseudorandom Correlated Generator (PCG). Their signing protocol consumes the least bandwidth among all schemes when amortized over large batches, but only supports full threshold, i.e., $t = n$.

Observing from [2] that the two multiplications in [46] can be computed in parallel instead of sequentially to reduce presigning rounds, Doerner et al. [32] proposed a three-round construction, subsuming their earlier scheme taking more rounds [31]. Using OT extension, their protocol is light on computation, but requires higher bandwidth. Cohen et al. [25] extend it to a 7-round protocol with identifiable abort.

Damgård et al. [28], Groth and Shoup [39], Katz and Urban [43], and Kondi and Ravi [45] study threshold ECDSA in the honest-majority setting. The scheme in [39] is the only one with mitigations to the attack surface analyzed in [40] among honest-majority and threshold-optimal ones. The drawback of these honest-majority schemes is that threshold signing requires at least twice as many participants as are needed to simply reconstruct the secret key.

We refer the reader to [32] and the updated full version of [16] for comprehensive information on other works.

1.3 Paper Organization

The rest of this paper is organized as follows. In section 2, we first recall ECDSA, then define the syntax and security of threshold signature schemes. Next, we introduce the cryptographic building blocks, namely, Non-Interactive Multiplication (NIM) and Non-Interactive Zero-Knowledge Arguments of Knowledge (NIZKAoK). Then, we outline the assumption on single-party ECDSA introduced in [4]. In section 3, we describe our two-round threshold ECDSA scheme using the building blocks and prove its security. In section 4, we outline how we instantiate the building blocks based on class groups. In section 5, we present performance metrics and analyze the efficiency of our proposed scheme.

2 Preliminaries

General Notation. Let κ and κ_{st} denote the computational and statistical security parameters, respectively. Let $\text{negl}(\kappa)$ denote a negligible function. We use “ $:=$ ” for deterministic assignment, “ $\leftarrow$ ” for assignment from a randomized algorithm, and “ $\xleftarrow{\$}$ ” for uniform random sampling. Groups are written in multiplicative notation. The integer interval $[m..n]$ represents $\{m, m+1, \dots, n\}$, and $[n]$ means $[1..n]$. Let PPT denote probabilistic polynomial time.

2.1 ECDSA

Let $\mathbb{G}$ denote the elliptic curve group generated by g of prime order q , and let $\mathbb{Z}_q$ denote the field $\mathbb{Z}/q\mathbb{Z}$ of integers modulo q . Let $H_{\text{sig}} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ be a cryptographic hash function.

DEFINITION 1 (ECDSA). *The Elliptic Curve Digital Signature Algorithm ECDSA = (Setup, KeyGen, Sign, Verify) is defined as follows:*

- $\text{Setup}(1^\kappa) \rightarrow \text{pp}$: On input the security parameter, the setup algorithm returns public parameters $\text{pp} = ((\mathbb{G}, q, g), H_{\text{sig}})$, which will be implicitly given as input to other algorithms.
- $\text{KeyGen}(\text{pp}) \rightarrow (X, x)$: On input the public parameters, the key generation algorithm samples a signing key $x \leftarrow \mathbb{Z}_q$, computes the verification key $X := g^x$, and returns the key pair (X, x) .
- $\text{Sign}(x, \text{msg}) \rightarrow (r, \sigma)$: On input the signing key x and the message msg , the signing algorithm computes $m := H_{\text{sig}}(\text{msg})$, samples a nonce $k \leftarrow \mathbb{Z}_q$, computes $R := g^k$, its x -coordinate $r := R|_{x\text{-axis}}$, and $\sigma := k^{-1}(m + rx) \bmod q$, and returns the signature (r, σ) .
- $\text{Verify}(X, \text{msg}, (r, \sigma)) \rightarrow \{0, 1\}$: On input the verification key X , the message msg , and the signature (r, σ) , the verification algorithm computes $m := H_{\text{sig}}(\text{msg})$, computes $\hat{R} := (g^m X^r)^{1/\sigma}$, and returns 1 if $\hat{R}|_{x\text{-axis}} = r$ and 0 otherwise.

2.2 Defining Threshold Signatures

We define the syntax of our scheme in a similar manner to recent works on threshold signatures.

DEFINITION 2 (TWO-ROUND THRESHOLD SIGNATURES). *A two-round threshold signature scheme $\text{TS} = (\text{Setup}, \text{KeyGen}, \text{Presign}, \text{Sign}, \text{Combine}, \text{Verify})$ has the following syntax:*

- $\text{Setup}(1^\kappa) \rightarrow \text{pp}$: On input the security parameter, the setup algorithm returns public parameters pp , which will be implicitly given as input to other algorithms.
 - $\text{KeyGen}(t, n) \rightarrow (\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]})$: On input the threshold t and the number of parties n , the probabilistic key generation algorithm outputs a verification key pk that represents the entirety of the n parties, the set $\{\text{pk}_i\}_{i \in [n]}$ of public keys for each party, and the set $\{\text{sk}_i\}_{i \in [n]}$ of secret keys for each party. Each sk_i is privately held by party i , and $(\text{pk}, \{\text{pk}_i\}_{i \in [n]})$ is made public.
 - $(\text{Presign}, \text{Sign}, \text{Combine})$: These three algorithms make up the signing protocol. Let $\mathbf{P} \subseteq [n]$ be a quorum of t or more parties. For each signing session, each party indexed by $i \in \mathbf{P}$ runs the following two algorithms:
 - $\text{Presign}(i) \rightarrow (\text{pm}_i^{(1)}, \text{pst}_i)$: The presigning algorithm returns the party's round-1 protocol message $\text{pm}_i^{(1)}$ and presigning secret state pst_i .
 - $\text{Sign}(i, \text{msg}, \{(\text{pk}_j, \text{pm}_j^{(1)})\}_{j \in \mathbf{P}}, \text{sk}_i, \text{pst}_i) \rightarrow \text{pm}_i^{(2)}$: On input the message msg to be signed, the other parties' public keys and round-1 protocol messages $\{(\text{pk}_j, \text{pm}_j^{(1)})\}_{j \in \mathbf{P}}$, and the party's secret key sk_i and presigning secret state pst_i , the signing algorithm returns the party's round-2 protocol message $\text{pm}_i^{(2)}$.
- Then, any entity can run the following algorithm:
- $\text{Combine}(\text{pk}, \text{msg}, \{(\text{pm}_j^{(1)}, \text{pm}_j^{(2)})\}_{j \in \mathbf{P}}) \rightarrow \text{sig}$: On input the verification key pk , the message msg to be signed, and the protocol messages $\{(\text{pm}_j^{(1)}, \text{pm}_j^{(2)})\}_{j \in \mathbf{P}}$ of parties in the quorum in both rounds, the deterministic combining algorithm returns a signature sig .
 - $\text{Verify}(\text{pk}, \text{msg}, \text{sig}) \rightarrow \{0, 1\}$: On input the verification key pk , the message msg , and the signature sig , the verification algorithm returns 1 if the signature is valid for the message msg and the verification key pk , and 0 otherwise.

Correctness can be intuitively defined: for all $\text{pp} \leftarrow \text{Setup}(1^\kappa)$ and $t \leq n$, after TS.KeyGen is run, for every quorum $\mathbf{P} \subseteq [n]$, $|\mathbf{P}| \geq t$ and every message msg , if each party indexed by $i \in \mathbf{P}$ honestly participates in the two rounds of signing and the protocol messages are delivered properly, then the signature output by TS.Combine always passes verification.

We now define the security with an experiment that adapts standard existential unforgeability under chosen-message attack (EUF-CMA) to the threshold setting.

DEFINITION 3 (STATIC UNFORGEABILITY). *Let TS be a two-round threshold signature scheme. For a PPT adversary $\mathcal{A}$, we consider the following experiment $\text{Exp}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}(\kappa, n, t, \mathbf{C})$:*

- Initialization Phase.
 - (1) Assert that the number of signers n , the threshold t , and the corruption set $\mathbf{C}$ satisfy $t \leq n$, $\mathbf{C} \subseteq [n]$, and $|\mathbf{C}| < t$.
 - (2) Let $\text{pp} \leftarrow \text{TS.Setup}(1^\kappa)$, and let $(\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]}) \leftarrow \text{TS.KeyGen}(t, n)$; send $(\text{pp}, \text{pk}, \{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_i\}_{i \in \mathbf{C}})$ to $\mathcal{A}$.
 - (3) Define the honest set $\mathbf{H} := [n] \setminus \mathbf{C}$.
 - (4) Initialize two lists $Q_{\text{presign}}, Q_{\text{sign}}$ as empty.
- Signing Phase. *The adversary $\mathcal{A}$ can adaptively query the following two oracles to engage honest parties in the signing protocol:*
 - Oracle $\text{PRESIGN}(i, \text{sid})$: On input the party index i and the session identifier sid from $\mathcal{A}$,
 - (1) Assert that $i \in \mathbf{H}$ and no item $(i, \text{sid}, \cdot, \cdot)$ exists in Q_{presign} .
 - (2) Let $(\text{pm}_i^{(1)}, \text{pst}_i) \leftarrow \text{TS.Presign}(i)$.
 - (3) Append $(i, \text{sid}, \text{pst}_i, \text{pm}_i^{(1)})$ to Q_{presign} .
 - (4) Return party i 's round-1 protocol message $\text{pm}_i^{(1)}$.
 - Oracle $\text{SIGN}(i, \text{sid}, \mathbf{P}, \text{msg}, \{\text{pm}_j^{(1)}\}_{j \in \mathbf{P}})$:
 - (1) Assert that the following hold:
 - (a) $\mathbf{P} \subseteq [n]$, $|\mathbf{P}| \geq t$, and $i \in \mathbf{P} \cap \mathbf{H}$;
 - (b) no item $(i, \text{sid}, \cdot, \cdot)$ exists in Q_{sign} ;
 - (c) $\forall j \in \mathbf{P} \cap \mathbf{H}$, an item $(j, \text{sid}, \cdot, \text{pm}_j^{(1)})$ exists in Q_{presign} ;
 - (d) no item $(\cdot, \text{sid}, \mathbf{P}', \text{msg}')$ exists in Q_{sign} such that $\mathbf{P}' \neq \mathbf{P}$ or $\text{msg}' \neq \text{msg}$.
 - (2) Append $(i, \text{sid}, \mathbf{P}, \text{msg})$ to Q_{sign} .
 - (3) Retrieve pst_i by looking up $(i, \text{sid}, \cdot, \cdot)$ in Q_{presign} .
 - (4) Return party i 's round-2 protocol message $\text{pm}_i^{(2)} \leftarrow \text{TS.Sign}(i, \text{msg}, \{(\text{pk}_j, \text{pm}_j^{(1)})\}_{j \in \mathbf{P}}, \text{sk}_i, \text{pst}_i)$.
- Outcome. *If $\mathcal{A}$ outputs $(\text{msg}^*, \text{sig}^*)$ such that no item $(\cdot, \cdot, \cdot, \text{msg}^*)$ exists in Q_{sign} and $\text{TS.Verify}(\text{pk}, \text{msg}^*, \text{sig}^*) = 1$, then the experiment returns 1. Otherwise, it returns 0.*

We say TS satisfies existential unforgeability under chosen-message attack with static corruption (or, static unforgeability) if, for all PPT adversary $\mathcal{A}$, the probability that the experiment $\text{Exp}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}$ returns 1 is negligible in κ ; i.e.,

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}(\kappa, n, t, \mathbf{C}) := \Pr[\text{Exp}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}(\kappa, n, t, \mathbf{C}) \Rightarrow 1] \leq \text{negl}(\kappa).$$

In the unforgeability game above, the adversary can initiate concurrent presigning and signing sessions, and wins if it outputs a valid forgery for a unsigned message (one that has not been queried to any honest party, which corresponds to TS-UF-0 in the hierarchy of definitions in [7]). We do not assume synchronous communication or broadcast channels; the adversary is allowed to arbitrarily interfere with the network and to send inconsistent

protocol messages to different honest parties. However, we do assume authenticated channels, as we require protocol messages from honest parties to be consistent; this treatment is in line with [5].

2.3 Non-Interactive Multiplication

We define the syntax of Non-Interactive Multiplication following Boyle et al. [12], with minor adjustments. Concrete instantiations of this primitive can be found in [3]. We provide details on a class-group-based instantiation in section 4.

DEFINITION 4 (NON-INTERACTIVE MULTIPLICATION). Let $\mathbb{Z}_q$ be the field of integers modulo q . A Non-Interactive Multiplication (NIM) scheme $\text{NIM} = (\text{Setup}, (\text{Encode}_\rho, \text{Decode}_\rho)_{\rho \in \{A, B\}})$ has the following syntax:

- $\text{Setup}(1^\kappa, q) \rightarrow \text{crs}$: On input the security parameter and the prime q , the setup algorithm returns a Common Reference String crs .
- $\text{Encode}_\rho(\text{crs}, v) \rightarrow (\text{pe}_\rho, \text{st}_\rho)$: On input crs and $v \in \mathbb{Z}_q$, the encoding algorithm, randomized and parameterized by a role $\rho \in \{A, B\}$, outputs a public encoding pe_ρ and a secret state st_ρ .
- $\text{Decode}_\rho(\text{crs}, \text{pe}_{\tilde{\rho}}, \text{st}_\rho) \rightarrow z_\rho$: On input crs , a public encoding $\text{pe}_{\tilde{\rho}}$ from a different role,¹ and a secret state st_ρ , the decoding algorithm, deterministic and parameterized by a role $\rho \in \{A, B\}$, outputs a share $z_\rho \in \mathbb{Z}_q$.

We require the following properties to hold:

Correctness: The scheme is correct if, for all $x, y \in \mathbb{Z}_q$,

$$\Pr \left[z_A + z_B = xy \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\kappa, q) \\ (\text{pe}_A, \text{st}_A) \leftarrow \text{Encode}_A(\text{crs}, x) \\ (\text{pe}_B, \text{st}_B) \leftarrow \text{Encode}_B(\text{crs}, y) \\ z_A := \text{Decode}(\text{crs}, \text{pe}_B, \text{st}_A) \\ z_B := \text{Decode}(\text{crs}, \text{pe}_A, \text{st}_B) \end{array} \right] = 1.$$

Input Privacy: The scheme is input-private if, for all PPT adversary $\mathcal{A}$ and role $\rho \in \{A, B\}$, the encoding hides the message; i.e.,

$$\text{Adv}_{\mathcal{A}, \text{NIM}}^{\text{priv-}\rho}(\kappa) := \Pr \left[b^* = b \mid \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\kappa, q) \\ (v_0, v_1, \text{st}) \leftarrow \mathcal{A}(\text{crs}) \\ b \leftarrow \{0, 1\} \\ (\text{pe}_\rho, \text{st}_\rho) \leftarrow \text{Encode}_\rho(\text{crs}, v_b) \\ b^* \leftarrow \mathcal{A}(\text{st}, \text{pe}_\rho) \end{array} \right] - \frac{1}{2} \leq \text{negl}(\kappa).$$

We will use that Encode_A is statistically hiding, though with caveats (see the remark in section 4.2).

2.4 NIZK Arguments of Knowledge

We define Non-Interactive Zero-Knowledge Arguments of Knowledge in the random oracle model.

DEFINITION 5 (NON-INTERACTIVE ZERO-KNOWLEDGE ARGUMENTS OF KNOWLEDGE). Let $\mathcal{L}$ be an NP language with associated relation $\mathcal{R}_{\mathcal{L}}$. Let $\mathcal{H}$ be a random oracle. A Non-Interactive Zero-Knowledge Argument of Knowledge scheme $\text{NIZKAoK} = (\text{Setup}, \text{Prove}^{\mathcal{H}}, \text{Verify}^{\mathcal{H}})$ has the following syntax:

- $\text{Setup}(1^\kappa) \rightarrow \text{pp}$: On input the security parameter, the setup algorithm returns public parameters pp , which will be implicitly given as input to other algorithms.

¹The notation $\tilde{\rho}$ denotes the role that is not ρ ; i.e., $\tilde{A} = B$ and $\tilde{B} = A$.

- $\text{Prove}^{\mathcal{H}}(\text{stmt}, \text{wit}) \rightarrow \pi$: On input a statement stmt and a witness wit such that $(\text{stmt}, \text{wit}) \in \mathcal{R}_{\mathcal{L}}$, the prover outputs a proof π .
- $\text{Verify}^{\mathcal{H}}(\text{stmt}, \pi) \rightarrow \{0, 1\}$: On input a statement stmt and proof π , the verifier outputs 1 for accept or 0 for reject.

We require the following properties to hold:

Completeness: The scheme is complete if, for all $(\text{stmt}, \text{wit}) \in \mathcal{R}_{\mathcal{L}}$, $\Pr[\text{Verify}^{\mathcal{H}}(\text{stmt}, \pi) = 1 \mid \pi \leftarrow \text{Prove}^{\mathcal{H}}(\text{stmt}, \text{wit})] = 1$.

Zero Knowledge: The scheme is zero-knowledge if, for all PPT $\mathcal{A}$, there exists a simulator $\text{Sim}^{\mathcal{H}}$ such that for all $(\text{stmt}, \text{wit}) \in \mathcal{R}_{\mathcal{L}}$,

$$\text{Adv}_{\mathcal{A}, \text{NIZKAoK}}^{\text{ZK}}(\kappa) := \left| \Pr[\mathcal{A}^{\mathcal{H}}(\text{stmt}, \pi) = 1 \mid \pi \leftarrow \text{Prove}^{\mathcal{H}}(\text{stmt}, \text{wit})] - \Pr[\mathcal{A}^{\mathcal{H}}(\text{stmt}, \pi) = 1 \mid \pi \leftarrow \text{Sim}^{\mathcal{H}}(\text{stmt})] \right| \leq \text{negl}(\kappa).$$

Knowledge Extractability: The scheme is knowledge-extractable if, for all PPT $\mathcal{A}$, there exists an extractor $\text{Extract}^{\mathcal{A}}$ such that

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{NIZKAoK}}^{\text{KE}}(\kappa) \\ := \Pr \left[\begin{array}{l} \text{Verify}^{\mathcal{H}}(\text{stmt}, \pi^*) = 1 \\ \wedge (\text{stmt}, \text{wit}^*) \notin \mathcal{R}_{\mathcal{L}} \end{array} \mid \begin{array}{l} (\text{stmt}, \pi^*) \leftarrow \mathcal{A}^{\mathcal{H}}(\text{pp}) \\ \text{wit}^* \leftarrow \text{Extract}^{\mathcal{A}}(\text{stmt}, \pi^*) \end{array} \right] \\ \leq \text{negl}(\kappa). \end{aligned}$$

2.5 Doubly-Enhanced Existential Unforgeability of ECDSA

We recall Doubly-Enhanced Existential Unforgeability of ECDSA, a computational assumption modeling the security of a variant of ECDSA from Adjedj et al. [4].

DEFINITION 6 (DOUBLY-ENHANCED EXISTENTIAL UNFORGEABILITY OF ECDSA [4]). For a PPT adversary $\mathcal{A}$, we consider the following experiment $\text{Exp}_{\mathcal{A}}^{\text{DEUF}}(\kappa)$:

- Initialization Phase.
 - (1) Let $\text{pp} \leftarrow \text{ECDSA.Setup}(1^\kappa)$, $(X, x) \leftarrow \text{ECDSA.KeyGen}(\text{pp})$; send (pp, X) to $\mathcal{A}$.
 - (2) Initialize three lists Q_{presign} , Q_{tweak} , and Q_{sign} as empty.
- Signing Phase. The adversary $\mathcal{A}$ can adaptively query the following three oracles:
 - Oracle $\text{PRESIGN}()$:
 - (1) Sample a nonce $k \leftarrow \mathbb{Z}_q$.
 - (2) Let $R := g^k$.
 - (3) Append (R, k) to Q_{presign} .
 - (4) Return the presignature R .
 - Oracle $\text{TWEAK}(R, z, m)$:
 - (1) Assert that $z, m \in \mathbb{Z}_q$, an item (R, k) exists in Q_{presign} , and no item $(R, z, m, \cdot)$ exists in Q_{tweak} nor $(R, \cdot)$ in Q_{sign} .
 - (2) Sample $\mu \leftarrow \mathbb{Z}_q$.
 - (3) Append (R, z, m, μ) to Q_{tweak} .
 - (4) Return the modifier μ .
 - Oracle $\text{SIGN}(R, z, m, \mu)$:
 - (1) Assert $(R, z, m, \mu) \in Q_{\text{tweak}}$ and no item $(R, \cdot)$ exists in Q_{sign} .
 - (2) Let $r := g^{zk+\mu}|_{x\text{-axis}}$, and let $\sigma := (zk + \mu)^{-1}(m + rx)$.
 - (3) Append (R, m) to Q_{sign} .
 - (4) Return σ .
- Outcome. If $\mathcal{A}$ outputs $(\text{msg}^*, (r^*, \sigma^*))$ such that $\text{ECDSA.Verify}(X, \text{msg}^*, (r^*, \sigma^*)) = 1$

and no item $(\cdot, H_{\text{sig}}(\text{msg}^*))$ exists in Q_{sign} , then the experiment returns 1. Otherwise, it returns 0.

We say that the Doubly-Enhanced Existential Unforgeability of ECDSA holds if, for all PPT adversary $\mathcal{A}$, the probability that the experiment $\text{Exp}_{\mathcal{A}}^{\text{DEUF}}$ returns 1 is negligible in κ ; i.e.,

$$\text{Adv}_{\mathcal{A}}^{\text{DEUF}}(\kappa) := \Pr[\text{Exp}_{\mathcal{A}}^{\text{DEUF}}(\kappa) \Rightarrow 1] \leq \text{negl}(\kappa).$$

The experiment defined in the assumption above is a variant of the standard EUF-CMA game, with some features related to presigning. In the experiment $\text{Exp}_{\mathcal{A}}^{\text{DEUF}}$, for each presignature $R = g^k$ provided by the challenger, the adversary can tweak it with a multiplicative factor z of its choice. For each tuple (R, z, m) queried, where m is a message hash, the challenger returns a modifier μ chosen uniformly at random. Before the adversary decides which (R, z, m) to actually use for signing, it can query the TWEAK oracle arbitrarily many times. For the signature queried at last, the challenger uses $zk + \mu$ as the nonce.

The experiment $\text{Exp}_{\mathcal{A}}^{\text{DEUF}}$ is an extension of the security experiment of ECDSA with re-randomized presignatures analyzed by Groth and Shoup [40, section 8], which is shown in the generic group model (GGM) [52] to be as secure as plain ECDSA without presigning. In the latter, the adversary is not allowed to tweak the presignature, or equivalently, z always equals 1; neither can $\mathcal{A}$ query modifiers with different message hashes for a single presignature. The added flexibility in $\text{Exp}_{\mathcal{A}}^{\text{DEUF}}$ allows us to use random oracles to derive re-randomizers in the protocol design. According to the analysis in the GGM by Adjedj et al. [4, theorem 2.1], this only incurs a constant security loss of 2^5 .

Despite what the terms suggest, the Doubly-Enhanced existential unforgeability is a weaker assumption than the Enhanced existential unforgeability [16, appendix E] underlying existing threshold ECDSA schemes with presigning. The latter assumes extra features on the hash function H_{sig} and has worse security. In that security experiment, the presignature is never re-randomized. For further information, we refer the reader to [40].

3 Two-Round Threshold ECDSA

In this section, we first describe our proposed two-round threshold ECDSA scheme, and then prove it secure assuming the security properties of the building blocks in the random oracle model.

3.1 Construction

We begin with a high-level explanation. First, the public parameters for ECDSA and the CRS for NIM are set up, and two hash functions to be used to re-randomize the presignature are given in the setup phase. Then, KeyGen generates the Shamir secret-shared signing key and the corresponding public verification key. In practice, this is done via a Distributed Key Generation (DKG) subprotocol, but to simplify presentation, we use a trusted dealer here. Additionally, each party publishes a NIM public encoding (under role B) of each secret-key share, with a NIZKAoK attesting that it is well-formed.

Each signing session consists of a presigning round and a signing round. In the presigning round, each party i sends to other parties an NIM public encoding of a random γ_i under role A , another one of a random k_i under role B , and g^{k_i} ; in addition, each party computes a NIZKAoK to prove the message is well-formed. After

receiving each other's message, the parties can use NIM decoding to obtain additive shares of products $k\gamma$ and $x\gamma$, where $k = \sum_i k_i$ and $\gamma = \sum_i \gamma_i$. While this is done in the presigning phase, we put these steps in the Sign procedure to fit the syntax we defined.

After the message msg to be signed is known, two hashes z, y are publicly derived from the message and the transcript of interaction. The presignature is re-randomized from g^k to $R = (g^k)^z g^y$. Each party i submits its shares of $w = m\gamma + rx\gamma$ and $u = y\gamma + zk\gamma$ to finalize signing, where $r = R|_{x\text{-axis}}$ and $m = H_{\text{sig}}(\text{msg})$.

In line with other works on threshld ECDSA, we assume authenticated communication. This can be realized by having each party sign each protocol message, binding it to a session identifier. Thus, the protocol implicitly requires a separate Public Key Infrastructure (PKI) and an external mechanism to produce session identifiers. They can be implemented using readily available resources, and we do not explicitly specify them here.

Threshold ECDSA Protocol

Setup: On input the security parameter 1^κ , the setup algorithm does:

- (1) Run $((\mathbb{G}, q, g), H_{\text{sig}}) \leftarrow \text{ECDSA.Setup}(1^\kappa)$;
- (2) Choose two hash functions $H_1, H_2 : \{0, 1\}^* \rightarrow \mathbb{Z}_q$;
- (3) Run $\text{crs} \leftarrow \text{NIM.Setup}(1^\kappa)$;
- (4) Sample $h \leftarrow \mathbb{G}$;
- (5) Return $\text{pp} := ((\mathbb{G}, q, g), H_{\text{sig}}, H_1, H_2, \text{crs}, h)$.

KeyGen: On input the threshold t and number of parties n , the key generation subprotocol first does:

- (1) Sample a random polynomial of degree $t - 1$ over $\mathbb{Z}_q$: $f(Z) = a_0 + a_1Z + \dots + a_{t-1}Z^{t-1}$;
- (2) Set $X := g^{a_0}$ as the verification key;
- (3) For each party indexed by $i \in [n]$, set $x_i := f(i)$ as its secret-key share, and set $X_i := g^{x_i}$ as the corresponding public-key share;
- (4) Send each x_i to party i privately, and make $(X, \{X_i\}_{i \in [n]})$ public.

Then, each party i does the following:

- (1) Compute $(\text{pe}_{x,i}, \text{st}_{x,i}) \leftarrow \text{NIM.Encode}_B(\text{crs}, x_i)$;
- (2) Compute $\pi_{x,i}$, a NIZKAoK that $\text{pe}_{x,i}$ and X_i use a consistent x_i ;
- (3) Broadcast $(\text{pe}_{x,i}, \pi_{x,i})$, and store $\text{st}_{x,i}$;
- (4) Abort if any other party broadcasts a NIZKAoK that fails verification.

Finally, the key generation subprotocol outputs the verification key $\text{pk} := X$, the parties' public keys $\{\text{pk}_i\}_{i \in [n]}$ where $\text{pk}_i := (X_i, \text{pe}_{x,i})$, and the parties' secret keys $\{\text{sk}_i\}_{i \in [n]}$ where $\text{sk}_i := (x_i, \text{st}_{x,i})$.

Presign: On receiving a presignature request, each party i does:

- (1) Sample $k_i, \gamma_i \leftarrow \mathbb{Z}_q$;
- (2) Compute:
 - $K_i := g^{k_i}$,
 - $\Gamma_i := g^{\gamma_i} h^{\rho_i}$ where $\rho_i \leftarrow \mathbb{Z}_q$;
 - $(\text{pe}_{k,i}, \text{st}_{k,i}) \leftarrow \text{NIM.Encode}_B(\text{crs}, k_i)$,
 - $(\text{pe}_{\gamma,i}, \text{st}_{\gamma,i}) \leftarrow \text{NIM.Encode}_A(\text{crs}, \gamma_i)$;

- (3) Compute π_i , a NIZKAoK proving that $\text{pe}_{k,i}$ and K_i use a consistent k_i and that $\text{pe}_{\gamma,i}$ and Γ_i use a consistent γ_i .

The presigning algorithm outputs the round-1 protocol message $\text{pm}_i^{(1)} := (K_i, \Gamma_i, \text{pe}_{k,i}, \text{pe}_{\gamma,i}, \pi_i)$ and the secret state $\text{pst}_i := (k_i, \gamma_i, \text{st}_{k,i}, \text{st}_{\gamma,i})$ of party i . It is assumed that each party i stores its secret state pst_i securely and sends the protocol message $\text{pm}_i^{(1)}$ to other parties authenticated and bound to a session identifier.

Sign: On receiving the presigning protocol messages $\{\text{pm}_j^{(1)}\}_{j \in \mathbf{P}}$ from parties in the quorum $\mathbf{P}$ where $\mathbf{P} \subseteq [n]$, $|\mathbf{P}| \geq t$ and a signing request for the message msg , each party indexed by $i \in \mathbf{P}$ does:

- (1) Set $K := \prod_{i \in \mathbf{P}} K_i$;
- (2) For each $j \in \mathbf{P} \setminus \{i\}$:
 - Parse $\text{pm}_j^{(1)}$ as $(K_j, \Gamma_j, \text{pe}_{k,j}, \text{pe}_{\gamma,j}, \pi_j)$;
 - Abort if the NIZKAoK π_j fails verification;
 - Compute:
 - $\alpha_{i,j} := \text{NIM.Decode}_B(\text{crs}, \text{pe}_{\gamma,j}, \text{st}_{k,i})$,
 - $\beta_{j,i} := \text{NIM.Decode}_A(\text{crs}, \text{pe}_{k,j}, \text{st}_{\gamma,i})$,
 - $\mu_{i,j} := \lambda_{i,\mathbf{P}} \cdot \text{NIM.Decode}_B(\text{crs}, \text{pe}_{\gamma,j}, \text{st}_{x,i})$,
 - $v_{j,i} := \lambda_{j,\mathbf{P}} \cdot \text{NIM.Decode}_A(\text{crs}, \text{pe}_{x,j}, \text{st}_{\gamma,i})$.

Here, $\lambda_{i,\mathbf{P}} := \prod_{j \in \mathbf{P} \setminus \{i\}} j \cdot (j-i)^{-1} \bmod q$ is the i -th Lagrange coefficient w.r.t. the quorum. The steps above can be performed before msg is known.

- (3) Compute:
 - $m := \text{H}_{\text{sig}}(\text{msg})$,
 - $z := \text{H}_1(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in \mathbf{P}})$,
 - $y := \text{H}_2(z)$;
- (4) Set $R := K^z g^y$, and $r := R|_{x\text{-axis}}$;
- (5) Set $w_i := m\gamma_i + r(\lambda_{i,\mathbf{P}}x\gamma_i + \sum_{j \in \mathbf{P} \setminus \{i\}} (\mu_{i,j} + v_{j,i}))$;
- (6) Set $u_i := y\gamma_i + z(k_i\gamma_i + \sum_{j \in \mathbf{P} \setminus \{i\}} (\alpha_{i,j} + \beta_{j,i}))$.

The signing algorithm outputs the round-2 protocol message $\text{pm}_i^{(2)} := (w_i, u_i)$ of party i . It is assumed that the presigning secret state pst_i is used only once and is erased after use.

Combine: On receiving $\{(\text{pm}_i^{(1)}, \text{pm}_i^{(2)})\}_{i \in \mathbf{P}}$, the combining algorithm computes r as specified above, and sets $w := \sum_{i \in \mathbf{P}} w_i$ and $u := \sum_{i \in \mathbf{P}} u_i$. It then computes $\sigma := w \cdot u^{-1} \bmod q$. It outputs the signature (r, σ) if $\text{ECDSA.Verify}(X, \text{msg}, (r, \sigma)) = 1$, and $\perp$ otherwise.

Correctness. We briefly outline the correctness of the protocol. By the correctness of NIM, for each pair of signers $i, j \in \mathbf{P}$, $i \neq j$, we have $\alpha_{i,j} + \beta_{i,j} = k_i\gamma_j$, and $\mu_{i,j} + v_{i,j} = (\lambda_{i,\mathbf{P}}x\gamma_i)\gamma_j$. Define $k := \sum_{i \in \mathbf{P}} k_i$, $\gamma := \sum_{i \in \mathbf{P}} \gamma_i$, and $x := \sum_{i \in \mathbf{P}} \lambda_{i,\mathbf{P}}x_i$. Rearranging the combining step, we have $w = \gamma \cdot (m + rx)$ and $u = \gamma \cdot (zk + y)$. Note that $R = g^{zk+y}$ and $r = R|_{x\text{-axis}}$. Therefore, we have $\sigma = w/u = (zk + y)^{-1}(m + rx)$, and we can conclude that (r, σ) is a valid signature on msg under the verification key X .

3.2 Security Analysis

THEOREM 1. Assume the security of NIM and NIZKAoK schemes, the Doubly-Enhanced Unforgeability of ECDSA, and that the hash

functions H_1, H_2 are modeled as random oracles. Then, the above threshold ECDSA protocol satisfies static unforgeability.

PROOF. Suppose there exists a PPT adversary $\mathcal{A}$ that wins the static unforgeability experiment $\text{Exp}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}$ w.r.t. our threshold ECDSA protocol with non-negligible advantage. We show that there exists a PPT algorithm $\mathcal{B}$ that can win the experiment of Doubly-Enhanced Unforgeability of ECDSA $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ with non-negligible probability, via a sequence of games that transition from the original static unforgeability game to the reduction.

Game G_0 . This is the original security game $\text{Exp}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}$ for our threshold signature scheme. The game honestly runs TS.Setup and TS.KeyGen , and answers queries to oracles PRESIGN and SIGN with honestly calculated presigning and online-signing protocol messages of the honest parties. The random oracles are instantiated using standard lazy sampling. By definition, we have

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}(\kappa, n, t, C) = \Pr[G_0 \Rightarrow 1].$$

Game G_1 . In this game, we replace the NIZKAoKs that honest parties calculate during key generation and presigning with simulated ones. Let $\mathcal{B}_1$ be an adversary to the zero-knowledge of NIZKAoK_{CL-DL}, and $\mathcal{B}'_1$ be one to that of NIZKAoK_{Ped}. By the hybrid argument, we have

$$\begin{aligned} |\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| &\leq |Q_{\text{presign}}| \cdot \text{Adv}_{\mathcal{B}'_1, \text{NIZKAoK}_{\text{Ped}}}^{\text{ZK}}(\kappa) \\ &\quad + (n + |Q_{\text{presign}}|) \cdot \text{Adv}_{\mathcal{B}_1, \text{NIZKAoK}_{\text{CL-DL}}}^{\text{ZK}}(\kappa). \end{aligned}$$

Game G_2 . This game is identical to G_1 , except that we extract corrupted parties' random choices of k_i, γ_i via the NIZKAoK extractors. Specifically, on each query to the random oracle H_1 with input $(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in \mathbf{P}})$, let $\text{Good} \in \{0, 1\}$ be 1 if and only if there exists a unique session sid such that, for all relevant honest parties $i \in \mathbf{P} \cap \mathbf{H}$, there is an item $(i, \text{sid}, \cdot, \text{pm}_i^{(1)})$ in Q_{presign} (i.e., the random oracle query matches exactly one possibly correct presigning session). If $\text{Good} = 1$, then for each relevant corrupted party $j \in \mathbf{P} \cap \mathbf{C}$, parse $\text{pm}_j^{(1)}$ as $(K_j, \Gamma_j, \text{pe}_{k,j}, \text{pe}_{\gamma,j}, \pi_j)$; if the NIZKAoK π_j verifies, then run the extractor to find k_j and γ_j .

The simulator aborts if extraction fails in any instance. Clearly, from $\mathcal{A}$'s view, G_2 differs to G_1 only when an abort happens. Let $\mathcal{B}_2$ and $\mathcal{B}'_2$ be adversaries to the knowledge extractability of the NIZKAoK schemes, and let Q_1 be the upper bound on the number of queries to H_1 . By the union bound, we have

$$\begin{aligned} |\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| &\leq Q_1 \cdot t \cdot (\text{Adv}_{\mathcal{B}_2, \text{NIZKAoK}_{\text{CL-DL}}}^{\text{KE}}(\kappa) \\ &\quad + \text{Adv}_{\mathcal{B}'_2, \text{NIZKAoK}_{\text{Ped}}}^{\text{KE}}(\kappa)). \end{aligned}$$

Game G_3 . In this game, we introduce a special rule for programming the random oracles H_1, H_2 that the protocol uses to rerandomize the presignature. Looking ahead, this will enable us to embed the modifier μ obtained from the doubly-enhanced ECDSA unforgeability game.

Specifically, on each query to H_1 with input $(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in \mathbf{P}})$, if $\text{Good} = 1$ and extraction of $\{k_j\}_{j \in \mathbf{P} \cap \mathbf{C}}$ succeeds in G_3 , then the simulator samples $\mu \leftarrow \mathbb{Z}_q$ and programs $H_2(z) := \mu - z \cdot \sum_{j \in \mathbf{P} \cap \mathbf{C}} k_j$ before returning $z = H_1(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in \mathbf{P}})$.

In this game, we rule out bad events that lead to programming conflicts, namely, if there is a hash collision of H_1 , or if the adversary

$\mathcal{A}$ guesses the random choice of z correctly and queries $H_2(z)$ before the simulator defines it in the special way. Then, because μ is uniformly distributed in $\mathbb{Z}_q$ and hidden from the adversary $\mathcal{A}$, the response of H_2 is still uniformly distributed. Let Q_2 be an upper bound on the number of queries to H_2 . By standard collision probability bounds,

$$|\Pr[G_2 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| \leq \frac{Q_1^2 + Q_1 Q_2}{q}.$$

Game G_4 . In this game, we make several changes as follows. First, in the key generation phase, the NIM public encoding $pe_{x,i}$ of each honest party i is no longer honestly derived using the Shamir share x_i of the secret key. Instead, we sample a random scalar $\hat{x}_i \leftarrow \mathbb{Z}_q$ and let $(pe_{x,i}, st_{x,i}) \leftarrow \text{NIM.Encode}_B(\text{crs}, \hat{x}_i)$. Second, when responding to oracle PRESIGN queries, the honest party i no longer uses a consistent random k_i when calculating K_i and $pe_{k,i}$. Instead, party i samples two random scalars $k_i, \hat{k}_i \leftarrow \mathbb{Z}_q$, then computes $K_i = g^{k_i}$ and $(pe_{k,i}, st_{k,i}) \leftarrow \text{NIM.Encode}_B(\text{crs}, \hat{k}_i)$. Looking ahead, these changes will be necessary for embedding X_i and K_i values with unknown discrete logarithms.

We must also fix the behavior of oracle SIGN so that the view remains consistent – namely, a correct execution of presigning and online signing should yield a valid signature; there is exactly one candidate, as we show below. Upon receiving a query to oracle SIGN with input $(i, \text{sid}, P, \text{msg}, \{\text{pm}_j^{(1)}\}_{j \in P})$ that passes the assertions, let $m := H_{\text{sig}}(\text{msg})$, $z := H_1(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in P})$, and $y := H_2(z)$ as in the protocol. Using the knowledge of the secret key x and the combined nonce $k = \sum_{i \in P} k_i$ that corresponds to the session sid, we immediately know the left part $r = g^{zk+y}|_{x\text{-axis}}$ and the right part $\sigma = (zk + y)^{-1}(m + rx)$ of the signature, and this is the only possible signature for this session sid.

We now show how to embed σ into the session. If this is the first time that the session sid is queried to oracle SIGN , then we let party i act as a special party. Sample $u' \leftarrow \mathbb{Z}_q$ and let $w' := \sigma \cdot u'$. Denote $\gamma = \sum_{j \in P} \gamma_j$. For corrupted parties $j \in P \cap C$, we define $\hat{x}_j = x_j$ and $\hat{k}_j = k_j$ (this is only to alleviate notation; note that the equalities do not hold for simulated honest parties $j \in P \cap H$). Let k' be such that $\gamma \cdot (zk' + y) = u'$ (hence $k' := z^{-1}(u' \gamma^{-1} - y)$), and then let $k'_i = k' - \sum_{j \neq i, j \in P} \hat{k}_j$. Similarly, let x' be such that $\gamma \cdot (m + rx') = w'$ (hence $x' := r^{-1}(w' \gamma^{-1} - m)$), and then let $x'_i = x' - \sum_{j \neq i, j \in P} \lambda_{j,P} \cdot \hat{x}_j$. The special party i 's simulated online-signing protocol message is thus (w'_i, u'_i) , where

$$\begin{aligned} w'_i &:= m\gamma_i + r(x'_i\gamma_i + \sum_{j \in P \setminus \{i\}} (\mu_{i,j} + v_{j,i} - \lambda_{i,P}\hat{x}_i\gamma_j + x'_i\gamma_j)), \\ u'_i &:= y\gamma_i + z(k'_i\gamma_i + \sum_{j \in P \setminus \{i\}} (\alpha_{i,j} + \beta_{j,i} - \hat{k}_i\gamma_j + k'_i\gamma_j)). \end{aligned}$$

The values $\alpha_{i,j}, \beta_{j,i}, \mu_{i,j}$ and $v_{j,i}$ above are honestly derived as in the protocol. Now, if sid has already been queried, i.e., the adversary is asking for round-2 protocol messages of other parties than the special one in the quorum, then the online-signing protocol steps are run honestly (except that $\hat{k}_i$ and $\hat{x}_i$ are used in place of k_i and x_i). It can be seen that the right part of the signature that the Combine algorithm outputs is $\sigma = w'/u'$, which is valid.

We now bound the distinguishing advantage between G_3 and G_4 . The presigning simulation is computationally indistinguishable as only NIM public encodings are changed. The online-signing

simulation fails if any of z, r and $\gamma = \sum_{i \in P} \gamma_i$ equals 0; however, the signing protocol would abort anyway in the first two cases, and the third also happens with negligible probability. Let $\mathcal{B}_4$ be an adversary to the input privacy of the NIM scheme. Since γ_i is statistically hidden by Encode_A (say with a statistical distance of δ) and perfectly hidden by the Pedersen commitment $g^{\gamma_i} h^{\rho_i}$, we have

$$\begin{aligned} |\Pr[G_3 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| &\leq 2(n + |Q_{\text{presign}}|) \cdot \text{Adv}_{\mathcal{B}_4, \text{NIM}}^{\text{priv-B}}(\kappa) \\ &\quad + |Q_{\text{sign}}|/q + |Q_{\text{presign}}| \cdot \delta. \end{aligned}$$

Game G_5 . In this game, we use the adversary $\mathcal{A}$ to construct a reduction $\mathcal{B}$ to the doubly-enhanced unforgeability of ECDSA. The changes on top of G_4 are as follows.

First, in the key generation phase, $\mathcal{B}$ receives the verification key X from $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$, and embeds the verification key X as the one output by KeyGen . In more detail, as we do not know the discrete logarithm of X , we give each corrupted party $j \in C$ a random secret-key share $x_j \leftarrow \mathbb{Z}_q$. The corresponding public-key share is $X_j = g^{x_j}$. For the honest parties, consistent public-key shares $\{X_i\}_{i \in H}$ with unknown discrete logarithms are simulated using standard Lagrange interpolation in the exponent.

Second, for each new session sid queried to oracle PRESIGN in $\text{Exp}_{\mathcal{A}, \text{TS}}^{\text{TSUF}}$ by $\mathcal{A}$ for the first time, let $\mathcal{B}$ query the oracle PRESIGN in $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$, obtaining a presignature R . When generating a presigning protocol message for this session sid, instead of sampling $k_i \leftarrow \mathbb{Z}_q$ and then computing $K_i = g^{k_i}$, each honest party i now samples $c_i \leftarrow \mathbb{Z}_q^*$ and computes $K_i = R^{c_i}$, where R is the presignature obtained from $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ that corresponds to this session sid.

Third, on each query to H_1 with input $(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in P})$, if $\text{Good} = 1$ and extraction of $\{k_j\}_{j \in P \cap C}$ succeeds in G_3 , then clearly we can identify a session sid as in G_3 . Let R be the presignature that $\mathcal{B}$ has obtained for this session sid. Let $z' := z \cdot \sum_{i \in P \cap H} c_i$, where $z = H_1(X, \text{msg}, \{\text{pm}_i^{(1)}\}_{i \in P})$ and $\{c_i\}_{i \in P \cap H}$ are aforementioned random numbers used for this session. The reduction $\mathcal{B}$ then queries oracle TWEAK in $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ with input (R, z', m) , where $m := H_{\text{sig}}(\text{msg})$, receiving μ in return. This game replaces the random choice of μ in G_3 with the one from oracle TWEAK as shown here. As a result, in the online phase corresponding to the queried transcript, the presignature will be re-randomized from $K = \prod_{i \in P} K_i$ to

$$K^z g^y = \left(\prod_{i \in P \cap H} R^{c_i} \cdot \prod_{j \in P \cap C} g^{k_j} \right)^z \cdot g^{\mu - z \cdot \sum_{j \in P \cap C} k_j} = R^{z'} g^{\mu},$$

where $y = H_2(z)$. Now that R is given by the $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ challenger, z' is the tweak induced by $\mathcal{B}$, and μ is the “modifier” induced by the $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ challenger, we are able to embed a re-randomized presignature, which the $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ challenger can use for signing, into a simulated possibly-correct threshold signing session.

Finally, the σ part of the signature that we use for simulating the oracle SIGN is no longer derived using the knowledge of k and x . Instead, it now comes from oracle SIGN in $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$; the reduction $\mathcal{B}$ simply queries oracle SIGN in $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ with input (R, z', m, μ) .

It is clear that G_5 is perfectly indistinguishable from G_4 , and the forgery by $\mathcal{A}$ can always be forwarded to the $\text{Exp}_{\mathcal{B}}^{\text{DEUF}}$ challenger to win the doubly-enhanced unforgeability game. Therefore,

$$\Pr[G_4 \Rightarrow 1] = \Pr[G_5 \Rightarrow 1] = \text{Adv}_{\mathcal{B}}^{\text{DEUF}}(\kappa).$$

Summarizing the above, we conclude that theorem 1 holds. $\square$

4 Concrete Instantiation

In this section, we provide information on how we instantiate our building blocks, namely NIM and accompanying NIZKAoKs. We do not claim novelty for results presented here, and refer the reader to original sources for their proofs.

Justifying the Choice. Abram et al. [3] show that NIM can be instantiated (without a non-negligible correctness error) from Decisional Composite Residuosity (DCR) in the Paillier group $\mathbb{Z}_{N^2}^*$, Quadratic Residuosity (QR) in the RSA group $\mathbb{Z}_N^*$, a variant of Joye–Libert, DDH-like assumptions in class groups, or LWE with a superpolynomial modulus-to-noise ratio.

We opt for an instantiation using class groups due to the following considerations. First, because we work in a malicious setting, the mismatching message space of Paillier/RSA instantiations lead to security problems that must be addressed using range proofs, which are rather expensive. This is evidenced by prior threshold ECDSA designs that use Paillier encryption with range proofs [37, 16]. In contrast, with the parameters set properly, class-group encryption can natively handle a message space of a field of integers modulo prime q , thus removing the need of range proofs. Second, class-group elements are much smaller than RSA/Paillier ones at the same level of security, mainly because the hardness of class-group assumptions does not require the hardness of factoring. Moreover, optimized class-group operations are faster than those in Paillier groups; this is contrary to earlier widely-held beliefs based on less efficient implementations. For information on the efficiency of class-group cryptography, we refer the reader to Bouvier et al. [11].

4.1 Background on Class Groups

Group Structure. We work in a finite abelian group $\hat{G}$. Inside $\hat{G}$, a cyclic subgroup F is generated by $f \in \hat{G}$ of order q , and discrete logarithms to the base f can be very efficiently computed. One can also sample elements in $\hat{G}$ that generate “hard” subgroups. Each hard subgroup is also cyclic, but it is hard to compute discrete logarithms in it or to find the order of it.

Distributed Dlog. Suppose two parties hold two correlated elements of the group $\hat{G}$, say Alice holds $f^z h$ and Bob holds h , where $z \in \mathbb{Z}_q$ and h is in a hard subgroup. There exists a Distributed Discrete Logarithm (DDLog) mechanism that allows them to obtain an additive sharing of z without any interaction; that is, Alice can obtain $z_A \in \mathbb{Z}_q$ and Bob can obtain $z_B \in \mathbb{Z}_q$ such that $z_A + z_B = z$.

The DDLog mechanism relies on a *coset labelling function* ϕ that, for each coset C of F , deterministically maps all elements in C to a specific one among them. More intuitively, for each h in a hard subgroup, let $C_h := \{f^x h \mid x \in \mathbb{Z}_q\}$, and there exists $\delta \in C_h$ such that for each $e \in C_h$ we have $\phi(e) = \delta$.

Continuing the description of DDLog: Alice computes $f^z h / \phi(f^z h)$, and Bob computes $\phi(h) / h$. It is straightforward to see that these results lie in the easy subgroup F . Calculating the discrete logarithms in F gives the two shares z_A and z_B , and they sum to z .

Under the Hood. The reader can find information on class-group cryptography in [11] and on the coset labelling function in [1]. Very roughly, the underlying set of $\hat{G}$ is squares in the class group of binary quadratic forms of discriminant $-pq^3$, where p is a random

1,571-bit prime at 128-bit security; the group operation is Gauss composition. The easy subgroup F is generated by the form $f = (q^2, q, \frac{1+pq}{4})$. A generator of a hard subgroup can be sampled as t^q , where t is randomly sampled in $\hat{G}$. All random coins used above can be made public to eliminate trapdoors, hence class groups are often said to have a transparent setup. The coset labelling function is a composition of algorithms 1 and 2 in [49].

Sampling Exponents. Let $H := \langle h \rangle$ be a hard subgroup of $\hat{G}$, where h is a generator sampled using the approach above. In cryptographic applications, we need a distribution $\mathcal{D}_q$ over integers such that $\{h^x \mid x \in \mathcal{D}_q\}$ is statistically close to the uniform distribution over the hard subgroup H . There are several ways to define $\mathcal{D}_q$ [21, lemma 4]. Typically, it is the uniform distribution over $[2^{\kappa_{\text{st}}} \tilde{s}]$, where κ_{st} is the statistical security parameter and $\tilde{s}$ is an upper bound on the order of the hard subgroup $|H|$. In practice, κ_{st} is set to 40 and $\tilde{s}$ is taken as $2^{\lceil \log \sqrt{pq} \rceil}$.

4.2 NIM from Class Groups

The Non-Interactive Multiplication construction from class groups, roughly speaking, uses Pedersen commitment for Encode_A , and Castagnos–Laguillaumie (CL) encryption [20, 21] for Encode_B . Earlier threshold ECDSA schemes have used CL encryption to realize multiplication via two-round OLE or threshold decryption. In contrast, our scheme does not require that each party samples a key pair in the class group; two publicly known random generators g_0, g_1 of hard subgroups are sufficient. The construction is as follows.

NIM construction

- **Setup**($1^\kappa, q$): Return $\text{crs} := (p, q, f, g_0, g_1)$, where g_0, g_1 are two random generators of hard subgroups.
- **Encode_A**(crs, v):
 - (1) Sample $s \leftarrow \mathcal{D}_q$;
 - (2) Compute $\text{pe}_A := g_0^s g_1^v$;
 - (3) Set $\text{st}_A := (s, v)$;
 - (4) Return $(\text{pe}_A, \text{st}_A)$.
- **Encode_B**(crs, v):
 - (1) Sample $r \leftarrow \mathcal{D}_q$;
 - (2) Compute $\text{pe}_B := (g_0^r, f^v g_1^r)$;
 - (3) Set $\text{st}_B := r$;
 - (4) Return $(\text{pe}_B, \text{st}_B)$.
- **Decode_A**($\text{crs}, \text{pe}_B, \text{st}_A$):
 - (1) Parse $(c_0, c_1) := \text{pe}_B$, $(s, v) := \text{st}_A$;
 - (2) Compute $e := c_0^s c_1^v$;
 - (3) Set $\delta := \phi(e)$;
 - (4) Set $z_A := \text{dlog}_f(e/\delta)$;
 - (5) Return z_A .
- **Decode_B**($\text{crs}, \text{pe}_A, \text{st}_B$):
 - (1) Parse $c := \text{pe}_A$, $r := \text{st}_B$;
 - (2) Compute $e := c^r$;
 - (3) Set $\delta := \phi(e)$;
 - (4) Set $z_B := \text{dlog}_f(\delta/e)$;
 - (5) Return z_B .

Correctness. The correctness of the NIM scheme above directly follows from [3, theorem 5]. We give an informal explanation here

to build some intuition. Suppose Alice has input $x \in \mathbb{Z}_q$ and Bob has input $y \in \mathbb{Z}_q$. They interact in a simultaneous round, as follows:

- (1) Alice sends $g_0^s g_1^x$ to Bob, and Bob sends $(g_0^r, f^y g_1^r)$ to Alice, where s, r are random numbers.
- (2) Alice computes $(g_0^s)^s (f^y g_1^r)^x = f^{xy} g_0^{sr} g_1^{xr}$, and Bob computes $(g_0^s g_1^x)^r = g_0^{sr} g_1^{xr}$, after receiving each other's message.
- (3) They compute a common label $\delta := \phi(f^{xy} g_0^{sr} g_1^{xr}) = \phi(g_0^{sr} g_1^{xr})$, where ϕ is the coset labelling function.
- (4) Alice gets an additive share $z_A := \text{dlog}_f(f^{xy} g_0^{sr} g_1^{xr} / \delta)$; Bob gets $z_B := \text{dlog}_f(\delta / g_0^{sr} g_1^{xr})$. At this point it is easy to check that $z_A + z_B = xy$.

Input Privacy. The Encode_A algorithm hides the input under the Uniformity assumption [3, definition 16]. The Encode_B algorithm hides the input under the Hard Subgroup Membership assumption [11, definition 2].

DEFINITION 7 (UNIFORMITY ASSUMPTION [3]). Let $\hat{G}$ be the class group, and let $t \leftarrow \hat{G}$, $h := t^q$ such that h is a hard-subgroup generator. The Uniformity assumption holds if, for all PPT $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}(c) = 1 \mid r \leftarrow \mathcal{D}_q, c := h^r] - \Pr[\mathcal{A}(c) = 1 \mid c \leftarrow \hat{G}] \right| \leq \text{negl}(\kappa).$$

DEFINITION 8 (HARD SUBGROUP MEMBERSHIP ASSUMPTION [11]). Let $\hat{G}$ be the class group, and let $t \leftarrow \hat{G}$, $h := t^q$ such that h is a hard-subgroup generator. The Hard Subgroup Membership assumption holds if, for all PPT $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}(c) = 1 \mid r \leftarrow \mathcal{D}_q, c := h^r] - \Pr[\mathcal{A}(c) = 1 \mid r \leftarrow \mathcal{D}_q, m \leftarrow \mathbb{Z}_q, c := f^m h^r] \right| \leq \text{negl}(\kappa).$$

THEOREM 2 (implicit in [3]). If the Uniformity and Hard Subgroup Membership assumptions hold in the class group $\hat{G}$, then the NIM scheme above satisfies Input Privacy (definition 4).

Remark. The uniformity assumption is introduced by [3] due to the consideration that the two hard-subgroup generators, independently chosen in the transparent setup setting, may not lie in the same cyclic subgroup; otherwise the Encode_A algorithm would clearly be statistically hiding. However, by the Cohen–Lenstra heuristics [24], the odd part of the class group in question is very likely cyclic, in which case all hard-subgroup generators are in the same cyclic subgroup. Our security proof assumes statistical hiding to slightly simplify the analysis. If in doubt, one can always use a multi-party ceremony to generate g_1 as a power of g_0 , but transparent independent generation should be equally secure.

4.3 Concrete NIZKAoKs

Alongside the NIM instantiation above, we need NIZKAoKs for the following relations:

$$\begin{aligned} \mathcal{R}_{\text{CL-DL}} &= \{(c_0, c_1, V); (r, v) \mid c_0 = g_0^r \wedge c_1 = f^v g_1^r \wedge V = g^v\}; \\ \mathcal{R}_{\text{Ped}} &= \{(c, V); (r, v, \rho) \mid c = g_0^r g_1^v \wedge V = g^v h^\rho\}. \end{aligned}$$

We briefly review relevant class-group Σ -protocol proof constructions, which have different design trade-offs because class groups have unknown orders. Using binary challenges lead to special-sound proofs [19], but the prover must be repeated many times, which incurs much overhead. Assuming *Low Order* (it is hard to find low-order elements in $\hat{G}$) and *Strong Root* (it is hard to find roots in $\hat{G}$ of random elements in a hard subgroup H), the Σ -protocol proofs

in [17] are proven knowledge-extractable. For these assumptions to hold, the hard-subgroup generators should not be adversarially chosen. Further, in cases where we do not need to extract exponents in hard subgroups for the security proof of the upper-level protocol to go through, standard soundness of class-group Σ -protocols can be simply proven under another *Rough Order* assumption [15, 14, 6], namely, assuming it is hard to distinguish between a random class group and one with rough order.

The security proof of our threshold ECDSA scheme requires the secrets x_i, k_i, γ_i of corrupted parties to be extracted from NIZKAoKs. The NIZKAoK obtained via compiling class-group Σ -protocols with the standard Fiat–Shamir transform is typically proven knowledge-extractable in the standalone setting via the forking argument (by rewinding the adversary) in the plain ROM. However, rewinding poses subtle issues in a concurrent protocol setting like ours. In the main security proof, we take a modular approach and simply assume that the NIZKAoKs are straight-line extractable, which is more or less an idealization. However, this can be justified because the exponents in the elliptic curve group $\mathbb{G}$ can be straight-line extracted if we use the Algebraic Group Model [35] or the Generic Group Model [52] for $\mathbb{G}$.

NIZKAoK construction

Let $H_{\text{FS}} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ be a hash function.

NIZKAoK_{CL-DL}:

- **Prove** $((c_0, c_1, V), (r, v))$:
 - (1) Sample $\tilde{r} \leftarrow [2^{2 \cdot \kappa_{\text{st}}} q \tilde{s}]$, $\tilde{v} \leftarrow \mathbb{Z}_q$.
 - (2) Compute $\tilde{c}_0 := g_0^{\tilde{r}}$, $\tilde{c}_1 := f^{\tilde{v}} g_1^{\tilde{r}}$, $\tilde{V} := g^{\tilde{v}}$.
 - (3) Compute $e := H_{\text{FS}}(g_0, g_1, f, g, c_0, c_1, V, \tilde{c}_0, \tilde{c}_1, \tilde{V})$.
 - (4) Compute $s_r := \tilde{r} + e \cdot r \in \mathbb{Z}$, $s_v := \tilde{v} + e \cdot v \bmod q$.
 - (5) Return $\pi := (\tilde{c}_0, \tilde{c}_1, \tilde{V}, s_r, s_v)$.
- **Verify** $((c_0, c_1, V), \pi)$:
 - (1) Parse $(\tilde{c}_0, \tilde{c}_1, \tilde{V}, s_r, s_v) := \pi$.
 - (2) Compute $e := H_{\text{FS}}(g_0, g_1, f, g, c_0, c_1, V, \tilde{c}_0, \tilde{c}_1, \tilde{V})$.
 - (3) Check that $s_r < 2^{2 \cdot \kappa_{\text{st}}} q \tilde{s}$ and $s_v \in \mathbb{Z}_q$.
 - (4) Check $g_0^{s_r} = \tilde{c}_0 \cdot c_0^e$, $f^{s_v} g_1^{s_r} = \tilde{c}_1 \cdot c_1^e$, $g^{s_v} = \tilde{V} \cdot V^e$.
 - (5) If all checks pass, output 1; otherwise, output 0.

NIZKAoK_{Ped}:

- **Prove** $((c, V), (r, v, \rho))$:
 - (1) Sample $\tilde{r} \leftarrow [2^{2 \cdot \kappa_{\text{st}}} q \tilde{s}]$, $\tilde{v} \leftarrow [2^{\kappa_{\text{st}}} q^2]$, $\tilde{\rho} \leftarrow \mathbb{Z}_q$.
 - (2) Compute $\tilde{c} := g_0^{\tilde{r}} g_1^{\tilde{v}}$, $\tilde{V} := g^{\tilde{v}} h^{\tilde{\rho}}$.
 - (3) Compute $e := H_{\text{FS}}(g_0, g_1, g, h, c, V, \tilde{c}, \tilde{V})$.
 - (4) Compute $s_r := \tilde{r} + e \cdot r \in \mathbb{Z}$, $s_v := \tilde{v} + e \cdot v \in \mathbb{Z}$, and $s_\rho := \tilde{\rho} + e \cdot \rho \in \mathbb{Z}_q$.
 - (5) Return $\pi := (\tilde{c}, \tilde{V}, s_r, s_v, s_\rho)$.
- **Verify** $((c, V), \pi)$:
 - (1) Parse $(\tilde{c}, \tilde{V}, s_r, s_v, s_\rho) := \pi$.
 - (2) Compute $e := H_{\text{FS}}(g_0, g_1, g, h, c, V, \tilde{c}, \tilde{V})$.
 - (3) Check that $s_r < 2^{2 \cdot \kappa_{\text{st}}} q \tilde{s}$, $s_v < 2^{\kappa_{\text{st}}} q^2$, and $s_\rho \in \mathbb{Z}_q$.
 - (4) Check that $g_0^{s_r} g_1^{s_v} = \tilde{c} \cdot c^e$ and $g^{s_v} h^{s_\rho} = \tilde{V} \cdot V^e$.
 - (5) If all checks pass, output 1; otherwise, output 0.

DEFINITION 9 (LOW ORDER ASSUMPTION [17]). *Let $\hat{G}$ be the class group. The Low Order assumption holds if, for all PPT $\mathcal{A}$,*

$$\Pr[\mu^d = 1 \in \hat{G} \wedge \mu \neq 1 \wedge |d| < 2^\kappa \mid (\mu, d) \leftarrow \mathcal{A}(\hat{G})] \leq \text{negl}(\kappa).$$

DEFINITION 10 (STRONG ROOT ASSUMPTION [17]). *Let H be a hard subgroup of the class group $\hat{G}$. The Strong Root assumption holds if, for all PPT $\mathcal{A}$,*

$$\Pr[X^e = Y \wedge \nexists k \in \mathbb{Z} \text{ s.t. } e = 2^k \mid Y \leftarrow H, (X, e) \leftarrow \mathcal{A}(Y)] \leq \text{negl}(\kappa).$$

THEOREM 3 ([17, 42]). *Assuming DL in $\mathbb{G}$, Low Order and Strong Root in $\hat{G}$, with H_{FS} modeled as a random oracle, the NIZKAoK scheme above is complete, zero-knowledge, and knowledge-extractable. Furthermore, exponents in the elliptic curve group $\mathbb{G}$ are straight-line extractable if $\mathbb{G}$ is modeled as a generic group.)*

A natural alternative that works in the plain ROM (thus removing the aforementioned idealization) is to use the Fischlin transform [33] to compile the elliptic-curve part of the Σ -protocol additionally. This will introduce 1–3 KiB of extra communication, depending on the parameters chosen, but the added computational cost will be less than 1% overall [22]. This approach is taken by the version of [46] that is used in production.

We summarize the above with a theorem statement for the security of the concrete instantiation.

THEOREM 4. *Assume that Uniformity, Hard Subgroup Membership, Low Order, and Strong Root hold in the class group $\hat{G}$, Doubly-Enhanced Unforgeability of ECDSA holds in the elliptic curve $\mathbb{G}$, hash functions H_1, H_2, H_{FS} are modeled as random oracles, and the elliptic curve $\mathbb{G}$ is modeled as a generic group (this is not needed if the Fischlin transform is used). Then, the concrete instantiation of our threshold ECDSA scheme satisfies static unforgeability.*

5 Implementation and Evaluation

We present performance metrics of our threshold ECDSA protocol, and compare it with prior work. We implement our protocol based on the BICYCL library [11] which provides class group operations. In line with other open-source threshold ECDSA implementations, we target 128-bit security, which covers usage with the Bitcoin curve secp256k1. The code is available at https://github.com/TECDSA/ecdsa_two_round.

5.1 Bandwidth Efficiency

We analyze the bandwidth usage in the signing protocol. Recall that the elliptic curve group is denoted $\mathbb{G}$ and the class group is denoted $\hat{G}$. In the presigning phase, each party sends two $\mathbb{G}$ elements and three $\hat{G}$ elements, excluding the NIZKAoKs. In the online signing phase, each party sends two $\mathbb{Z}_q$ elements. At 128-bit computational security and 40-bit statistical security, each $\hat{G}$ element takes 220 bytes, each $\mathbb{G}$ element takes 33 bytes, and each $\mathbb{Z}_q$ element takes 32 bytes. Therefore, communication excluding the proof is 790 bytes.

Flexible NIZKAoK Bandwidth. The size of the NIZKAoK sent in the presigning phase is calculated separately, because several variants may be considered in different practical settings. With the default formulation, each NIZKAoK_{CL-DL} proof consists of two $\hat{G}$ elements, a $\mathbb{G}$ element, a $\mathbb{Z}_q$ element, and an integer that is 157 bytes long; each NIZKAoK_{Ped} proof consists of a $\hat{G}$ element, a $\mathbb{G}$

element, a $\mathbb{Z}_q$ element, a 157-byte integer, and a 69-byte integer. Hence, the default size of the proof is 1,173 bytes.

One can utilize the Schnorr signature size reduction technique to significantly reduce the proof size. Namely, for a Σ -protocol transcript (A, e, s) , where e is derived from a hash function via the Fiat–Shamir heuristic, one can take (e, s) instead of (A, s) as the NIZKAoK proof. To verify such a proof, the first-round commitment A is reverse-sampled according to the verification equation, and if the Fiat–Shamir hash digest equals e , then the proof is accepted. Note that reverse sampling the first-round commitment is fast in both $\mathbb{G}$ and $\hat{G}$, and therefore the computation to verify such a proof is equal to that in the default formulation done naively. In this case, the proof takes only 479 bytes.

If presigning is performed in a known batch size, then one can also utilize batch proofs [6]. In this setting, one proof can be used to verify the well-formedness of m presigning protocol messages with a size that only increases logarithmically with m .

We report the most conservative number, $790 + 1173 = 1963$ bytes, as the total message length each party sends in each signing session, and do not delve into further details.

Comparison. If protocol messages are relayed by centralized servers, the total outbound communication of a party in our signing protocol is 1.9 KiB. This model is seen in threshold Schnorr schemes, e.g. [7], and we believe it accounts for many practical deployments. In this model, among other threshold ECDSA schemes, only [54] similarly achieves a constant communication of 3.4 KiB. Others all take linear communication per party due to pairwise OLE.

However, among prior threshold ECDSA papers, the common choice has been to measure communication in the peer-to-peer network model. In this case, all schemes require outbound communication per party that is at least linear in the number of parties. For our protocol, the number is $1.9 \cdot (t - 1)$ KiB.

We compare each party’s outbound communication with each peer among different threshold ECDSA schemes in table 2, following [16, figure 1]. Note that [16] gives estimates using 112-bit security for Paillier encryption (2048-bit modulus N). Since 128-bit security (3072-bit modulus) is practically used in well-known open-source implementations,² we also list numbers at 128-bit security accordingly. For schemes that use broadcasts, our estimates take broadcasts as multicasts and do not count the overhead, which favors those schemes.

It is generally agreed that schemes based on OT consume more bandwidth than those based on homomorphic encryption, and schemes based on Paillier consume more bandwidth than those based on Castagnos–Laguillaumie class group encryption. Our protocol instantiated with class groups can be seen as one that extends the last technical approach by getting rid of the OLE response messages. In the peer-to-peer model, our scheme approximately halves communication compared to some most bandwidth-efficient schemes [17, 18]. If a relaying server is available, our bandwidth savings would even be demonstrated asymptotically. Furthermore, as is explained in [4], a total communication of just 1.9 KiB makes our scheme very suitable for scenarios with constrained communication such as NFC, QR codes, and push notifications.

²For example <https://github.com/LFDT-Lockness/cgmp21>.

Table 2: Outbound communication with each peer among threshold ECDSA schemes at 128-bit security.

Scheme	Tool	Comm.	Note
GG18 [37]	Paillier enc	11.7 KiB 16.7 KiB	$\kappa = 112$ $\kappa = 128$
LN18 [46]	Paillier enc	7.8 KiB	$\kappa = 112$
		10.8 KiB	$\kappa = 128$
	OT	190 KiB 60 KiB	original ver. updated ver.
CGGMP20 [16]	Paillier enc	16 KiB	4-round, $\kappa = 112$
		23 KiB	4-round, $\kappa = 128$
		15.8 KiB	7-round, $\kappa = 112$
		22.8 KiB	7-round, $\kappa = 128$
DKLs19 [31]	OT	90 KiB	
DKLs24 [32]	OT	50 KiB	
CCLST20 [17]	CL enc	3.5 KiB	
CCLST23 [18]	CL enc	4.2 KiB	
WMC24 [54]	threshold CL enc	3.4 KiB	
ANOSS22 [2]	ring-LPN PCG	0.017n + 0.18 KiB *	
This work	class group NIM	1.9 KiB	

*[2] reports comm. amortized over a batch of 94019 signatures; their scheme only supports full threshold ($t = n$) and is not threshold-optimal. Some listed schemes require broadcasts, the overhead of which is not counted.

The scheme of Abram et al. [2] is the only one that consumes less bandwidth than ours, based on reported communication amortized over 94019 signing sessions. Its limitations are heavier computation, unimplemented preprocessing protocol, and only supporting full threshold (i.e., $t = n$) and not being threshold-optimal.

5.2 Computational Efficiency

We measure computational cost in terms of single-threaded running time of each party in a signing session. The machine used for the benchmarks features Intel i7-13700K CPU and 32 GB of memory, and runs Ubuntu 24.04. Results are presented in table 3.

Table 3: Single-threaded signer computation time in milliseconds among threshold ECDSA schemes at 128-bit security.

Scheme	3-party	5-party	7-party	Note
CGGMP20 [16]	918	1810	2720	four-round ver.
CCLST20 [17]	388	628	868	
DKLs24 [32]	21	41	61	
WMC24 [54]	281	411	535	
This work	224	405	586	

Each number is the combined running time of presigning and signing. Based on local benchmarking and not accounting for network latency.

The schemes listed in table 3 represent the major practical approaches used for secure computation, namely Paillier, OT, and class groups. Furthermore, in practical cryptocurrency wallets, the

number of parties are typically 3 to 5 and rarely exceeds 10, and we thus focus on such small-scale deployments in table 3. We remark that in decentralized finance applications there can sometimes be more than 20 parties; however, as the listed schemes all have linear computation, the performance can be extrapolated in this case.

The computation of schemes based on OT, e.g. DKLs24 [32], primarily consists of symmetric cryptographic operations (via OT extension techniques) and can benefit from existing highly-optimized implementations. Therefore, it is rather hard for schemes like ours to outperform them computationally. However, for [16, 32] and our scheme, the overwhelming majority of computation is in the presigning phase, and the online-signing round takes less than 1 millisecond of computation; hence, with presigning, the difference in computation would not be pronounced for end users. Even when the protocol falls back to interactive signing (without available presignatures) our scheme would be fast enough. Threshold ECDSA is often used in applications with a human factor, such as cryptocurrency wallets where the user must check the transaction before signing. The average human reaction time to a visual stimulus is around 250 milliseconds, and any latency below that is generally acceptable. Anyway, since ours requires one fewer round and much less communication, it will outperform theirs in certain scenarios.

Our scheme runs faster than CGGMP20 [16] by roughly 4x, mainly because [16] relies on computationally expensive range proofs, which other schemes based on Paillier encryption also do. Recent works [56, 53] report improvements to them by optimizing the range proofs; their reported speedups are less than 2x, and we estimate that ours would still outperform these improved variants. At higher security levels, the advantage of schemes based on class groups over Paillier ones will be greater; see [11, table 3].

Our scheme outperforms CCLST20 [17] by 30% to 40%, and is basically on par with WMC24 [54]. However, for WMC24, a significant proportion of computation happens in the online-signing round, because their last step is threshold decryption, which requires class group exponentiations linear in n . With similar total computational costs, it is certainly preferable to offload the bulk of the computation to the presigning phase, which ours does. Moreover, WMC24 takes 4 rounds, 3 of which are broadcasts. We have not implemented CCLST23 [18], which is the identifiable-abort version of CCLST20 [17], and WMYC23 [55], but [54] reports that both are at least 2x slower than WMC24.

The ANOSS22 [2] scheme requires a large batch of presigning material to be expanded from preshared seeds at once. They report that, at a batch size of 94019, each signature takes more than 1 second of computation, and the total time for the whole batch is quite high. Their scheme appears suitable for scenarios where computing power is ample but communication is extremely constrained.

Further Optimizations. Our protocol implementation does not include many in-depth optimizations. However, there are clear-cut paths towards speedups. In our scheme, the computation that increases linearly with the number of parties consists in NIM decoding and NIZKAoK verification, and the main bottleneck is class-group exponentiation. Beaugrand et al. [6] report a speedup of 4.7x for fixed-basis class-group exponentiation at a cost of 1.2 seconds of precomputation. Using this strategy, NIZKAoK verification can be much faster since it mainly requires fixed-basis exponentiations

base g_0 and g_1 . Besides, hand-written assembly may lead to faster class-group operations than BICYCL [11] which is based on GMP.

6 Conclusion

We propose the first two-round threshold ECDSA scheme in the threshold-optimal setting, solving an important open question in this line of research. It outperforms the state of the art in bandwidth efficiency, and its computational speed is also competitive.

We leave a few questions for future research. First, our protocol is proven secure under the static corruption model. For threshold Schnorr-like signatures, there have been advances in achieving security under adaptive corruption, but for threshold ECDSA, full adaptive security (without relying on the single-inconsistent-player model) remains an open challenge. Second, while our template is compatible with other number-theoretic tools such as Paillier, it remains to find an alternative approach that also leads to a two-round solution and is both light in computation (comparably to OT extension) and compact in communication.

Acknowledgments

We thank the CCS 2025 reviewers for their constructive feedback. This work was supported by the National Key Research and Development Program of China (2022YFB2701700), the National Natural Science Foundation of China (62472255, 62302271), a Shandong Provincial Youth Innovation Team grant (2024KJH79), the Shandong Provincial Natural Science Foundation (ZR2023MF045, ZR2023QF088), and the Qingdao Natural Science Foundation (23-2-1-152-zyyd-jch). Hong-Sheng Zhou was supported in part by NSF grant CNS-1801470 and a VCU Research Quest grant.

References

- [1] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. 2022. An algebraic framework for silent preprocessing with trustless setup and active security. In *CRYPTO 2022* (LNCS 13510). Springer, 421–452. doi:10.1007/978-3-031-15985-5_15.
- [2] Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. 2022. Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In *IEEE S&P 2022*, 2554–2572. doi:10.1109/SP46214.2022.9833559.
- [3] Damiano Abram, Lawrence Roy, and Peter Scholl. 2024. Succinct homomorphic secret sharing. In *EUROCRYPT 2024* (LNCS 14656). Springer, 301–330. doi:10.1007/978-3-031-58751-1_11.
- [4] Michael Adjedj, Constantin Blokh, Geoffroy Couteau, Arik Galansky, Antoine Joux, and Nikolaos Makriyannis. 2024. Two-round 2PC ECDSA at the cost of 1 OLE. Cryptology ePrint Archive, Paper 2024/1950. (2024).
- [5] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. 2025. Glacius: threshold Schnorr signatures from DDH with full adaptive security. In *EUROCRYPT 2025* (LNCS 15602). Springer, 304–334. doi:10.1007/978-3-031-91124-8_11.
- [6] Agathe Beaugrand, Guilhem Castagnos, and Fabien Laguillaumie. 2025. Efficient succinct zero-knowledge arguments in the CL framework. *Journal of Cryptology*, 38, 1, 13. doi:10.1007/s00145-024-09534-1.
- [7] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. 2022. Better than advertised security for non-interactive threshold signatures. In *CRYPTO 2022* (LNCS 13510). Springer, 517–550. doi:10.1007/978-3-031-15985-5_18.
- [8] Constantin Blokh, Nikolaos Makriyannis, and Udi Peled. 2022. Efficient asymmetric threshold ECDSA for MPC-based cold storage. Cryptology ePrint Archive, Paper 2022/1296. (2022).
- [9] Dan Boneh, Iftach Haitner, Yehuda Lindell, and Gil Segev. 2025. Exponent-VRFs and their applications. In *EUROCRYPT 2025* (LNCS 15607). Springer, 195–224. doi:10.1007/978-3-031-91098-2_8.
- [10] Alexandre Bouez and Kalpana Singh. 2023. One round threshold ECDSA without roll call. In *CT-RSA 2023* (LNCS 13871). Springer, 389–414. doi:10.1007/978-3-031-30872-7_15.
- [11] Cyril Bouvier, Guilhem Castagnos, Laurent Imbert, and Fabien Laguillaumie. 2023. I want to ride my BICYCL: BICYCL implements cryptography in class groups. *Journal of Cryptology*, 36, 3, 17. doi:10.1007/s00145-023-09459-1.
- [12] Elette Boyle, Lalita Devadas, and Sacha Servan-Schreiber. 2025. Non-interactive distributed point functions. In *PKC 2025* (LNCS 15674). Springer, 3–35. doi:10.1007/978-3-031-91820-9_1.
- [13] Elette Boyle, Niv Gilboa, and Yuval Ishai. 2016. Breaking the circuit size barrier for secure computation under DDH. In *CRYPTO 2016* (LNCS 9814). Springer, 509–539. doi:10.1007/978-3-662-53018-4_19.
- [14] Lennart Braun, Guilhem Castagnos, Ivan Damgård, Fabien Laguillaumie, Kelsey Melissaris, Claudio Orlandi, and Ida Tucker. 2024. An improved threshold homomorphic cryptosystem based on class groups. In *SCN 2024* (LNCS 14974). Springer, 24–46. doi:10.1007/978-3-031-71073-5_2.
- [15] Lennart Braun, Ivan Damgård, and Claudio Orlandi. 2023. Secure multiparty computation from threshold encryption based on class groups. In *CRYPTO 2023* (LNCS 14081). Springer, 613–645. doi:10.1007/978-3-031-38557-5_20.
- [16] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. 2020. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In *ACM CCS 2020*. ACM, 1769–1787. doi:10.1145/3372297.3423367.
- [17] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. 2020. Bandwidth-efficient threshold ECDSA. In *PKC 2020* (LNCS 12111). Springer, 266–296. doi:10.1007/978-3-030-45388-6_10.
- [18] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. 2023. Bandwidth-efficient threshold ECDSA revisited: online/offline extensions, identifiable aborts proactive and adaptive security. *Theoretical Computer Science*, 939, 78–104, C. doi:10.1016/j.tcs.2022.10.016.
- [19] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. 2019. Two-party ECDSA from hash proof systems and efficient instantiations. In *CRYPTO 2019* (LNCS 11694). Springer, 191–221. doi:10.1007/978-3-030-26954-8_7.
- [20] Guilhem Castagnos and Fabien Laguillaumie. 2015. Linearly homomorphic encryption from DDH. In *CT-RSA 2015* (LNCS 9048). Springer, 487–505. doi:10.1007/978-3-319-16715-2_26.
- [21] Guilhem Castagnos, Fabien Laguillaumie, and Ida Tucker. 2018. Practical fully secure unrestricted inner product functional encryption modulo p . In *ASIACRYPT 2018* (LNCS 11273). Springer, 733–764. doi:10.1007/978-3-030-03329-3_25.
- [22] Yi-Hsiu Chen and Yehuda Lindell. 2024. Optimizing and implementing Fischlin’s transform for UC-secure zero knowledge. *Communications in Cryptology*, 1, 2. doi:10.62056/a66chey6b.
- [23] Megan Chen, Carmit Hazay, Yuval Ishai, Yuriy Kashnikov, Daniele Micciancio, Tarik Riviere, abhi shelat, Muthu Venkatasubramanian, and Ruihan Wang. 2021. Diogenes: lightweight scalable RSA modulus generation with a dishonest majority. In *IEEE S&P 2021*, 590–607. doi:10.1109/SP40001.2021.00025.
- [24] H. Cohen and H. W. Lenstra Jr. 1984. Heuristics on class groups of number fields. In *Number Theory Noordwijkerhout 1983*. Springer, 33–62. doi:10.1007/BFb0099440.
- [25] Ran Cohen, Jack Doerner, Yashvanth Kondi, and Abhi Shelat. 2024. Secure multiparty computation with identifiable abort via vindicating release. In *CRYPTO 2024* (LNCS 14927). Springer, 36–73. doi:10.1007/978-3-031-68397-8_2.
- [26] Ran Cohen and Yehuda Lindell. 2017. Fairness versus guaranteed output delivery in secure multiparty computation. *Journal of Cryptology*, 30, 4, 1157–1186. doi:10.1007/s00145-016-9245-5.
- [27] Anders P. K. Dalskov, Claudio Orlandi, Marcel Keller, Kris Shrivash, and Haya Shulman. 2020. Securing DNSSEC keys via threshold ECDSA from generic MPC. In *ESORICS 2020* (LNCS 12309). Springer, 654–673. doi:10.1007/978-3-030-59013-0_32.
- [28] Ivan Damgård, Thomas Pelle Jakobsen, Jesper Buus Nielsen, Jakob Illeborg Pagter, and Michael Bækvang Østergaard. 2020. Fast threshold ECDSA with honest majority. In *SCN 2020* (LNCS 12238). Springer, 382–400. doi:10.1007/978-3-030-57990-6_19.
- [29] Yi Deng, Shunli Ma, Xinxuan Zhang, Hailong Wang, Xuyang Song, and Xiang Xie. 2021. Promise Σ -protocol: how to construct efficient threshold ECDSA from encryptions based on class groups. In *ASIACRYPT 2021* (LNCS 13093). Springer, 557–586. doi:10.1007/978-3-030-92068-5_19.
- [30] Yvo Desmedt and Yair Frankel. 1990. Threshold cryptosystems. In *CRYPTO ’89* (LNCS 435). Springer, 307–315. doi:10.1007/0-387-34805-0_28.
- [31] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. 2019. Threshold ECDSA from ECDSA assumptions: the multiparty case. In *IEEE S&P 2019*, 1051–1066. doi:10.1109/SP.2019.00024.
- [32] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. 2024. Threshold ECDSA in three rounds. In *IEEE S&P 2024*, 3053–3071. doi:10.1109/SP54263.2024.00178.
- [33] Marc Fischlin. 2005. Communication-efficient non-interactive proofs of knowledge with online extractors. In *CRYPTO 2005* (LNCS 3621). Springer, 152–168. doi:10.1007/11535218_10.
- [34] Offir Friedmann, Avichai Marmor, Dolev Mutzari, Omer Sadika, Yehonatan C. Scaly, Yuval Spiizer, and Avishay Yanai. 2024. 2PC-MPC: emulating two party ECDSA in large-scale MPC. Cryptology ePrint Archive, Paper 2024/253. (2024).

- [35] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. 2018. The algebraic group model and its applications. In *CRYPTO 2018* (LNCS 10992). Springer, 33–62. doi:10.1007/978-3-319-96881-0_2.
- [36] Adam Gągól, Jędrzej Kula, Damian Straszak, and Michał Świątek. 2020. Threshold ECDSA for decentralized asset custody. *Cryptology ePrint Archive*, Paper 2020/498. (2020).
- [37] Rosario Gennaro and Steven Goldfeder. 2018. Fast multiparty threshold ECDSA with fast trustless setup. In *ACM CCS 2018*. ACM, 1179–1194. doi:10.1145/3243734.3243859.
- [38] Rosario Gennaro, Steven Goldfeder, and Arvind Narayanan. 2016. Threshold-optimal DSA/ECDSA signatures and an application to bitcoin wallet security. In *ACNS 2016* (LNCS 9696). Springer, 156–174. doi:10.1007/978-3-319-39555-5_9.
- [39] Jens Groth and Victor Shoup. 2022. Design and analysis of a distributed ECDSA signing service. *Cryptology ePrint Archive*, Paper 2022/506. (2022).
- [40] Jens Groth and Victor Shoup. 2022. On the security of ECDSA with additive key derivation and presignatures. In *EUROCRYPT 2022* (LNCS 13275). Springer, 365–396. doi:10.1007/978-3-031-06944-4_13.
- [41] Dominik Hartmann and Eike Kiltz. 2023. Limits in the provable security of ECDSA signatures. In *TCC 2023* (LNCS 14372). Springer, 279–309. doi:10.1007/978-3-031-48624-1_11.
- [42] Aniket Kate, Easwar Vivek Mangipudi, Pratyay Mukherjee, Hamza Saleem, and Sri Aravinda Krishnan Thyagarajan. 2024. Non-interactive VSS using class groups and application to DKG. In *ACM CCS 2024*. ACM, 4286–4300. doi:10.1145/3658644.3670312.
- [43] Jonathan Katz and Antoine Urban. 2025. Honest-majority threshold ECDSA with batch generation of key-independent presignatures. *Communications in Cryptology*, 2, 1. doi:10.62056/a0l5wol7.
- [44] Chelsea Komlo and Ian Goldberg. 2020. FROST: flexible round-optimized Schnorr threshold signatures. In *SAC 2020* (LNCS 12804). Springer, 34–65. doi:10.1007/978-3-030-81652-0_2.
- [45] Yashvanth Kondi and Divya Ravi. 2025. Separating broadcast from cheater identification. In *ACM CCS 2025*. ACM. doi:10.1145/3719027.3765041.
- [46] Yehuda Lindell and Ariel Nof. 2018. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In *ACM CCS 2018*. ACM, 1837–1854. doi:10.1145/3243734.3243788.
- [47] Yingjie Lyu, Zengpeng Li, Hong-Sheng Zhou, Haiyang Xue, Mei Wang, Shuchao Wang, and Mengling Liu. 2025. Bandwidth-efficient robust threshold ECDSA in three rounds. *Cryptology ePrint Archive*, Paper 2025/828. (2025).
- [48] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. 2021. The rise of Paillier: homomorphic secret sharing and public-key silent OT. In *EUROCRYPT 2021* (LNCS 12696). Springer, 678–708. doi:10.1007/978-3-030-77870-5_24.
- [49] Sachar Paulus and Tsuyoshi Takagi. 2000. A new public-key cryptosystem over a quadratic order with quadratic decryption time. *Journal of Cryptology*, 13, 2, 263–272. doi:10.1007/s001459910010.
- [50] Michaela Pettit. 2021. Efficient threshold-optimal ECDSA. In *CANS 2021* (LNCS 13099). Springer, 116–135. doi:10.1007/978-3-030-92548-2_7.
- [51] Lawrence Roy and Jaspal Singh. 2021. Large message homomorphic secret sharing from DCR and applications. In *CRYPTO 2021* (LNCS 12827). Springer, 687–717. doi:10.1007/978-3-030-84252-9_23.
- [52] Victor Shoup. 1997. Lower bounds for discrete logarithms and related problems. In *EUROCRYPT '97* (LNCS 1233). Springer, 256–266. doi:10.1007/3-540-69053-0_18.
- [53] Guofeng Tang, Shuai Han, Li Lin, Changzheng Wei, and Ying Yan. 2024. Batch range proof: how to make threshold ECDSA more efficient. In *ACM CCS 2024*. ACM, 4256–4270. doi:10.1145/3658644.3670287.
- [54] Harry W. H. Wong, Jack P. K. Ma, and Sherman S. M. Chow. 2024. Secure multiparty computation of threshold signatures made more efficient. In *NDSS 2024*. doi:10.14722/ndss.2024.24601.
- [55] Harry W. H. Wong, Jack P. K. Ma, Hoover H. F. Yin, and Sherman S. M. Chow. 2023. Real threshold ECDSA. In *NDSS 2023*. doi:10.14722/ndss.2023.24817.
- [56] Zhikang Xie, Mengling Liu, Haiyang Xue, Man Ho Au, Robert H. Deng, and Siu-Ming Yiu. 2024. Direct range proofs for Paillier cryptosystem and their applications. In *ACM CCS 2024*. ACM, 899–913. doi:10.1145/3658644.3690261.
- [57] Haiyang Xue, Man Ho Au, Mengling Liu, Kwan Yin Chan, Handong Cui, Xiang Xie, Tsz Hon Yuen, and Chengru Zhang. 2023. Efficient multiplicative-to-additive function from Joye-Libert cryptosystem and its application to threshold ECDSA. In *ACM CCS 2023*. ACM, 2974–2988. doi:10.1145/3576915.3616595.
- [58] Tsz Hon Yuen, Handong Cui, and Xiang Xie. 2021. Compact zero-knowledge proofs for threshold ECDSA with trustless setup. In *PKC 2021* (LNCS 12710). Springer, 481–511. doi:10.1007/978-3-030-75245-3_18.